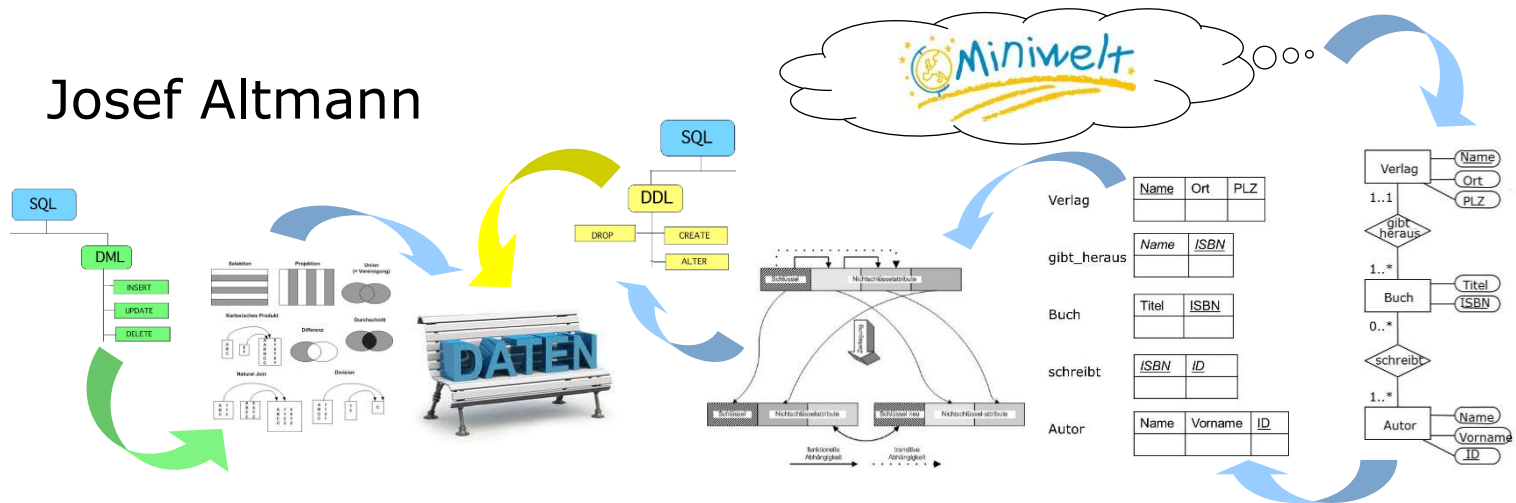


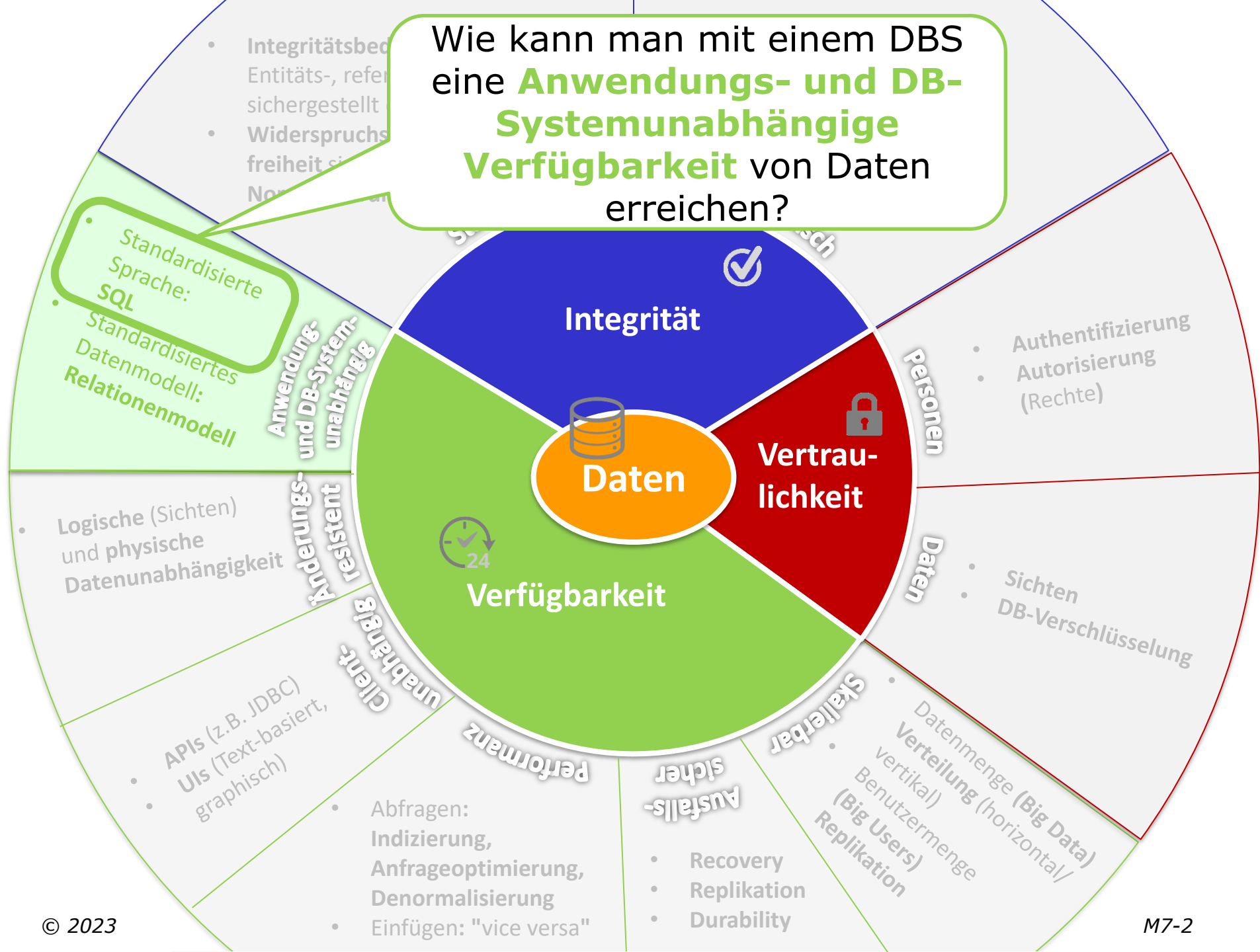
Modul 7

SQL-Datenabfragesprache

Josef Altmann



Wie kann man mit einem DBS eine **Anwendungs- und DB-Systemunabhängige Verfügbarkeit** von Daten erreichen?



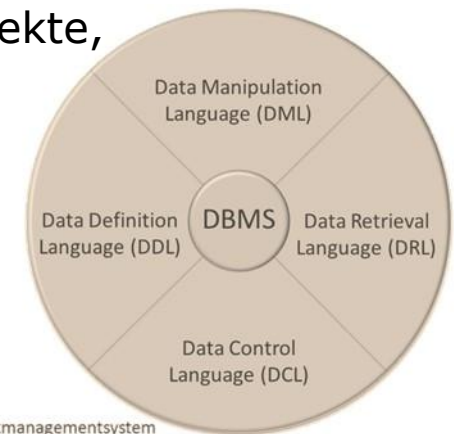
Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte

Structured Query Language

- SQL stellt eine standardisierte
 - **Datendefinitionssprache** (DDL)
 - Erstellen, Ändern und Löschen von Datenbankobjekten (Tabellen, etc.)
 - **Datenmanipulationssprache** (DML)
 - Einfügen, Ändern und Löschen von Daten
 - **Anfrage (Query)-Sprache** (DRL)
 - Abfragen von Daten (dies ist der komplexeste Teil von SQL)
 - **Kontrollsprache** (DCL)
 - Verwalten von Zugriffsrechten auf Datenbankobjekte, Transaktionskontrolle, etc.

bereit.



Datenabfragesprache

Structured Query Language

- **SQL** wurde auf Basis von relationaler Algebra entwickelt
 - **deklarative** Sprache, d.h., es wird formuliert **WAS** man haben möchte, aber nicht **WIE** es berechnet werden soll
 - **mengenorientierte** Sprache
- **Abarbeitung** durch **Übersetzung** der Abfrage mittels Parser in **relationale Algebra** und Optimierung durch **Anfrageoptimierer** des DBMS
- **Relationen** werden in Form von Tabellen dargestellt

Überblick

■ Einführung Datenabfragesprache SQL

■ Datenabfragesprache SQL

- SQL-Kern
- Einfache SQL-Anfragen
- Anfragen über mehrere Relationen
- Mengenoperationen
- Aggregatfunktionen
- Aggregate und Gruppierung
- Geschachtelte Anfragen
- Existenziell quantifizierte Anfragen
- Allquantifizierte Anfragen
- Nullwerte
- Spezielle Sprachkonstrukte

1. Teil

SQL-Kern

Auswertungsreihenfolge einen SFW-Blocks

Auswertungsreihenfolge

3

SELECT

- Projektionsliste (= Ergebnisschema)
- arithmetische Operationen und Aggregatfunktionen

1

FROM

- zu verarbeitende Relationen, evtl. Umbenennungen

2

WHERE

- Selektions-, Verbundbedingungen
- Geschachtelte Anfragen (wieder ein SFW-Block)

SQL-Kern

Anfrage

■ Struktur einer einfachen Anfrage:

```
SELECT attribute  
FROM tabelle  
WHERE bedingungen
```


$$\pi_{\text{attribute}}(\sigma_{\text{bedingungen}}(\text{tabelle}))$$

■ Attribute

- Liste der Attribute, die ausgegeben werden

■ Tabelle

- Name der Tabelle, die durchsucht wird

■ Bedingung

- Kriterium, das jedes ausgegebene Tupel erfüllen muss

■ Ergebnis einer SQL-Anfrage ist eine (virtuelle) Relation.

SQL-Kern

Anfrage

Beispiel

Geben Sie Personalnummer und Name aller C4-Professoren an:

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

```
SELECT PersNr, Name  
FROM Professor  
WHERE Rang = 'C4';
```

} $\pi_{\text{PersNr, Name}}(\sigma_{\text{Rang} = 'C4'}(\text{Professor}))$

Ergebnis

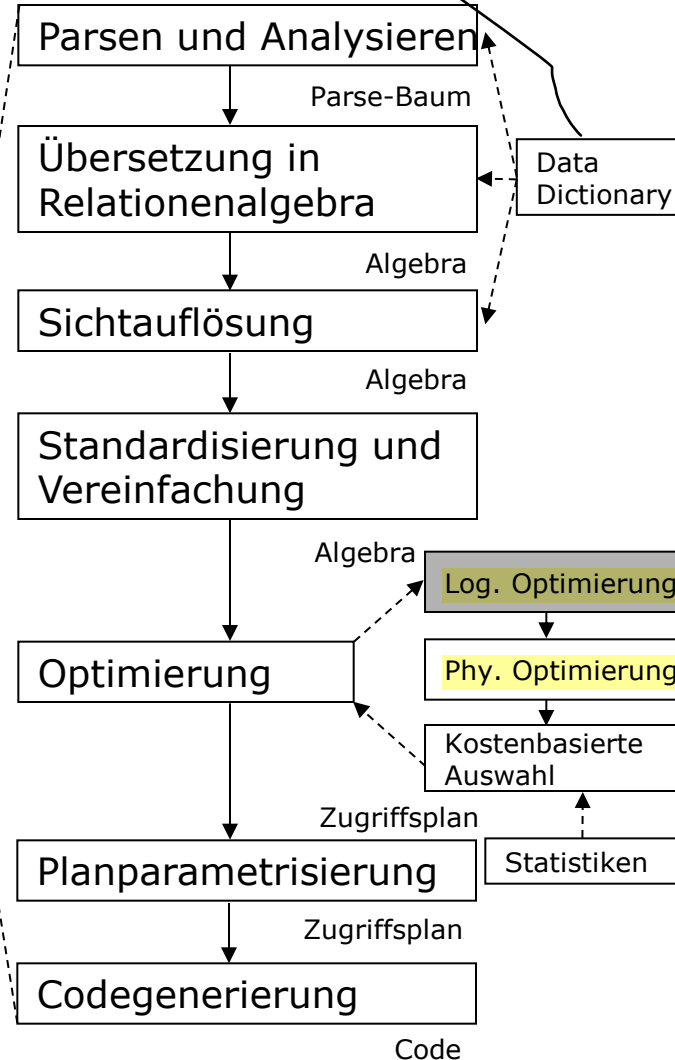
PersNr	Name
2125	Sokrates
2126	Russel
2136	Curie
2137	Kant

SQL-Kern

Anfragebearbeitung

DD enthält alle wichtigen Informationen, d.h. welche Tabellen gibt es, welche Attr., etc.

SQL-Datenabfragesprache



Anfrage

Übersetzer

Ausführungsplan

Laufzeitsystem

Ergebnis

```

SELECT PrsNr, Name
FROM Professor
WHERE Rang = 'C4';
    
```

OPERATION	OBJECT_NAME	OPTIONS	COST
SELECT STATEMENT			3
TABLE ACCESS	PROFESSOREN	FULL	3
Filterprädikate			
RANG='C4'			

	PERSNR	NAME
1	2125	Sokrates
2	2126	Russel
3	2136	Curie
4	2137	Kant

- ⇒ Zentrale Aufgabe der Anfragebearbeitung ist die Übersetzung der deklarativen Anfrage in einen effizienten, prozeduralen Auswertungsplan!
- ⇒ **Query Optimizer** führt diese Aufgabe durch.

SQL-Kern

Eigenheiten von SQL

- SQL ist **case-insensitive**, d.h.
Vorname=vorname=VORNAME=VorName
- Innerhalb von **Anführungszeichen** ist SQL **case-sensitive**, d.h. Rang='C4' $\neq$ Rang ='c4'
- Jeder Befehl wird mit einem **Strichpunkt** abgeschlossen
- **Kommentarzeilen** werden in /* ... */ eingeschlossen (auch über mehrere Zeilen), oder mit -- und REM eingeleitet (kommentiert den Rest der Zeile aus)
- **Unterschiede zu relationaler Algebra:**
 - Selektion, Projektion und Verbund sind keine Einzeloperatoren sondern in SFW-Block eingebettet
 - SQL-Ergebnisrelationen sind im allg. nicht duplikatfrei, d.h. die gleiche Zeile kann mehrmals vorkommen (Multimengen)
 - Zusätzliche Auswertungsmöglichkeiten (Aggregate, Sortieren)

Relationale Universitätsdatenbank

Student

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Professor

<u>PersNr</u>	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Assistent

<u>PersNr</u>	Name	Fachgebiet	<u>Boss</u>
3002	Platon	Ideenlehre	2125
3003	Aristoteles	Syllogistik	2125
3004	Wittgenstein	Sprachtheorie	2126
3005	Rhetikus	Planetenbewegung	2127
3006	Newton	Kepler Gesetze	2127
3007	Spinoza	Gott und Natur	2126

Vorlesung

<u>VorlNr</u>	Titel	SWS	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

voraussetzen

<u>Vorgaenger</u>	<u>Nachfolger</u>
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

hoeren

<u>MatrNr</u>	<u>VorlNr</u>
26120	5001
27550	5001
27550	4052
28106	5041
28106	5001
28106	4052
28106	4630
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

pruefen

<u>MatrNr</u>	<u>VorlNr</u>	<u>PersNr</u>	Note
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

Einfache SQL-Anfragen

Auswahl von Tabellen – Die FROM-Klausel

Einfachste Form

```
SELECT *
FROM tabelle;
```

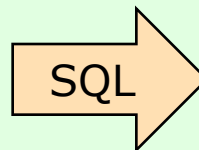
- **FROM**-Klausel enthält die Relationen, die in der Anfrage involviert sind.
- Umbenennung notwendig, wenn die gleiche Relation mehrmals im **FROM**-Teil vorkommt.

Beispiel

```
SELECT *
FROM Professor;
```

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7



Ergebnis

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Einfache SQL-Anfragen

Auswahl von Spalten– Die **SELECT**-Klausel

Festlegung der Projektionsattribute

```
SELECT [ALL | DISTINCT]  
  { attribut |  
    arithmetischer-ausdruck |  
    aggregat-funktion }*  
FROM ...;
```

- **SELECT**-Klausel bestimmt die Spalten einer Relation, die ausgegeben werden sollen
- ***** gibt an, dass alle Spalten ausgegeben werden
- Arithmetische Ausdrücke
 - über Attribute dieser Relationen
- Aggregat- bzw. Gruppierungsfunktionen
 - über Attribute dieser Relationen
- **DISTINCT**
 - Ergebnismenge statt Multimenge, d.h. es erfolgt eine Duplikatelimination

Einfache SQL-Anfragen

DISTINCT eliminiert Duplikate

Keine Duplikatelimination

```
SELECT ALL Rang  
FROM Professor;
```



Ergebnis

Rang
C4
C4
C3
C3
C3
C4
C4

- liefert die Ergebnismenge als Multimenge

Duplikatelimination

Geben Sie alle Rangbezeichnungen für Professoren ohne Duplikate aus.

```
SELECT DISTINCT Rang  
FROM Professor;
```



Ergebnis

Rang
C4
C3

- Achtung: Duplikateliminierung ist "teuer" (Sortieren + Eliminieren)

Einfache SQL-Anfragen

Aliasnamen für Spalten

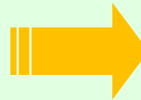
- Wird direkt **nach** dem **Spaltennamen** angegeben
- **Optional** kann zwischen dem Spalten- und dem Aliasnamen das Schlüsselwort **AS** angegeben werden

Spaltenüberschrift

```
SELECT PersNr AS Personalnummer, Name "Familiennome"  
FROM Professor;
```

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7



Ergebnis

PERSONALNUMMER	Familiennome
2125	Sokrates
2126	Russel
2127	Kopernikus
2133	Popper
2134	Augustinus
2136	Curie
2137	Kant

Einfache SQL-Anfragen

Sortierung

Sortierung

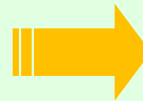
ORDER BY attribut1 [**DESC**|**ASC**] [, attribut1 [**DESC**|**ASC**]]...

Beispiel

```
SELECT PersNr, Name, Rang
FROM Professor
ORDER BY Rang DESC, Name ASC;
```

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7



Ergebnis

PersNr	Name	Rang
2136	Curie	C4
2137	Kant	C4
2126	Russel	C4
2125	Sokrates	C4
2134	Augustinus	C3
2127	Kopernikus	C3
2133	Popper	C3

- **ORDER BY**-Klausel steht am Ende der **SELECT**-Klausel
- Sortierung aufsteigend (**ASC**, Default) oder absteigend (**DESC**)
- Ohne Angabe der **ORDER-BY**-Klausel wird die Reihenfolge der Ausgabe durch das System bestimmt!

Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Kartesisches Produkt
 - Allgemeiner Verbund
 - Natürlicher Verbund
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte

Anfragen über mehrere Relationen

Kartesisches Produkt

- Bei mehr als einer Relation in der **FROM**-Klausel wird mit **CROSS JOIN** das kartesische Produkt gebildet

Beispiel: CROSS JOIN

```
SELECT Name, Titel
FROM Professor CROSS JOIN Vorlesung;
```

Vorlesung

VorINr	Titel	SWS	GelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
...	...	...	...
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

10 Tupel

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

7 Tupel

Ergebnis

VorINr	Titel	SWS	GelesenVon	PersNr	Name	Rang	Raum
5001	Grundzüge	4	2137	2125	Sokrates	C4	225
5001	Grundzüge	4	2137	2126	Russel	C4	232
5001	Grundzüge	4	2137	2127	Kopernikus	C3	310
...	...	...	...	...	...	...	...
4630	Die 3 Kritiken	4	2137	2134	Augustinus	C3	309
4630	Die 3 Kritiken	4	2137	2136	Curie	C4	36
4630	Die 3 Kritiken	4	2137	2137	Kant	C4	7

70 Tupel

* =

- alle Kombinationen werden ausgegeben!

Beispiel: implizit mit **alter** Schreibweise

```
SELECT Name, Titel
FROM Professor, Vorlesung;
```

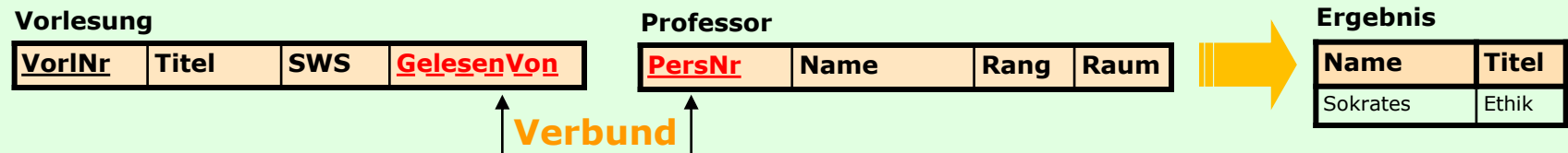
Anfragen über mehrere Relationen

Allgemeiner Verbund

■ Verbund mit beliebigen Bedingungen

Beispiel

Welche Professoren lesen die Vorlesung Ethik?
Informationen aus Tabellen: Professoren, Vorlesungen



Anfragen über mehrere Relationen

Allgemeiner Verbund

- Steht die gewünschte Information in mehreren Tabellen, so müssen diese in der **WHERE**-Klausel verknüpft werden.
- Struktur einer Abfrage über mehrere Tabellen:

Anfrage über mehrere Relationen

```
SELECT attribut  
FROM tabelle1 CROSS JOIN tabelle2 CROSS JOIN tabelleN  
[ WHERE bedingungen ];
```

- **bedingungen:** greifen auf die Attribute der verschiedenen Tabellen zu
- **Auswertungsreihenfolge:**
 1. Bilde Kreuzprodukt der FROM-Tabellen
 2. Überprüfe für jede Zeile die WHERE-Bedingungen und wähle passende aus
 3. Projiziere auf SELECT-Attribute

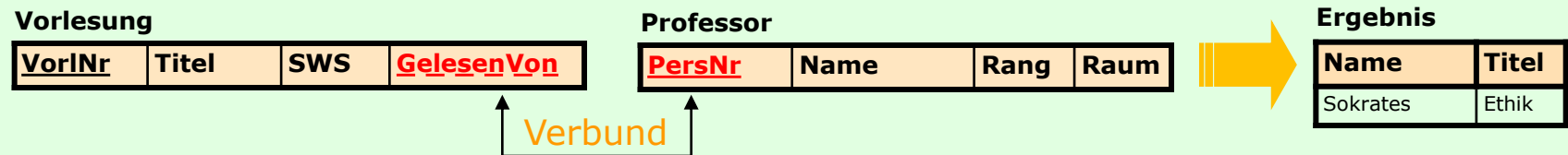
Anfragen über mehrere Relationen

Allgemeiner Verbund

■ Verbund mit beliebigen Bedingungen

Beispiel

Welche Professoren lesen die Vorlesung Ethik?
Informationen aus Tabellen: Professoren, Vorlesungen



```
SELECT  Name, Titel
FROM    Professor CROSS JOIN Vorlesung
WHERE   PersNr = gelesenVon AND
        Titel = 'Ethik';
```

Anfragen über mehrere Relationen

Allgemeiner Verbund – Abarbeitung

Professor

<u>PersNr</u>	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Vorlesung

<u>VorlNr</u>	Titel	SWS	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

FROM Professor **CROSS JOIN** Vorlesung

Anfragen über mehrere Relationen

Allgemeiner Verbund – Abarbeitung



FROM Professor **CROSS JOIN** Vorlesung

PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	GelesenVon
2125	Sokrates	C4	226	5001	Grundzüge	4	2137
2125	Sokrates	C4	226	5041	Ethik	4	2125
...	...	...	...	...	...	...	...
2125	Sokrates	C4	226	5049	Maeeutik	2	2125
...	...	...	...	...	...	...	...
2137	Kant	C4	7	4630	Die drei Kritiken	4	2137



WHERE PersNr = gelesenVon **AND** Titel = 'Ethik'

PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	GelesenVon
2125	Sokrates	C4	226	5041	Ethik	4	2125



SELECT Name, Titel

Name	Titel
Sokrates	Ethik

Anfragen über mehrere Relationen

Allgemeiner Verbund – INNER JOIN

- Verbund mit beliebiger Bedingung – **ON**-Klausel

Verbund: CROSS JOIN

```
SELECT Name, Titel
FROM Professor CROSS JOIN Vorlesung
WHERE PersNr = gelesenVon AND Titel = 'Ethik';
```

Verbund: INNER JOIN mit ON-Klausel (ergibt Teilmenge des CROSS JOIN)

```
SELECT Name, Titel
FROM Professor INNER JOIN Vorlesung
ON (PersNr = gelesenVon) AND Titel = 'Ethik';
```

Equi-Join ← , <, <=, >=, >, <> als Operatoren zulässig

Anfragen über mehrere Relationen

Natürlicher Verbund – Verbund über gleichnamige Spalten

Beispiel

Welche Studierende besuchen welche Vorlesungen?

Student

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Vorlesung

<u>VorlNr</u>	Titel	SWS	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

hoeren

<u>MatrNr</u>	<u>VorlNr</u>
26120	5001
27550	5001
27550	4052
28106	5041
28106	5001
28106	4052
28106	4630
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

Anfragen über mehrere Relationen

Natürlicher Verbund – Verbund über gleichnamige Spalten

■ Verbund über gleichnamige Spalten – **ON**-Klausel

Welche Studierende besuchen welche Vorlesungen?

```
SELECT *
FROM Student INNER JOIN hoeren ON (Student.MatrNr = hoeren.MatrNr)
INNER JOIN Vorlesung ON ( hoeren.VorlNr = Vorlesung.VorlNr)
```

(Student ⋈_{Student.MatrNr=hoeren.MatrNr} hoeren) ⋈ _{hoeren.VorlNr=Vorlesung.VorlNr} Vorlesung

MatrNr	Name	Semester	MatrNr_1	VorlNr	VorlNr_1	Titel	SWS	GelesenVon
26120	Fichte	10	26120	5001	5001	Grundzüge	4	2137
27550	Schopenhauer	6	27550	5001	5001	Grundzüge	4	2137
27550	Schopenhauer	6	27550	4052	4052	Logik	4	2125
28106	Carnap	3	28106	5041	5041	Ethik	4	2125
28106	Carnap	3	28106	5001	5001	Grundzüge	4	2137
28106	Carnap	3	28106	4052	4052	Logik	4	2125
28106	Carnap	3	28106	4630	4630	Die drei Kritiken	4	2137
29120	Theophrastos	2	29120	5001	5001	Grundzüge	4	2137
29120	Theophrastos	2	29120	5041	5041	Ethik	4	2125
29120	Theophrastos	2	29120	5049	5049	Mäeutik	2	2125
29555	Feuerbach	2	29555	5022	5022	Glaube und Wissen	2	2134
25403	Jonas	12	25403	5022	5022	Glaube und Wissen	2	2134

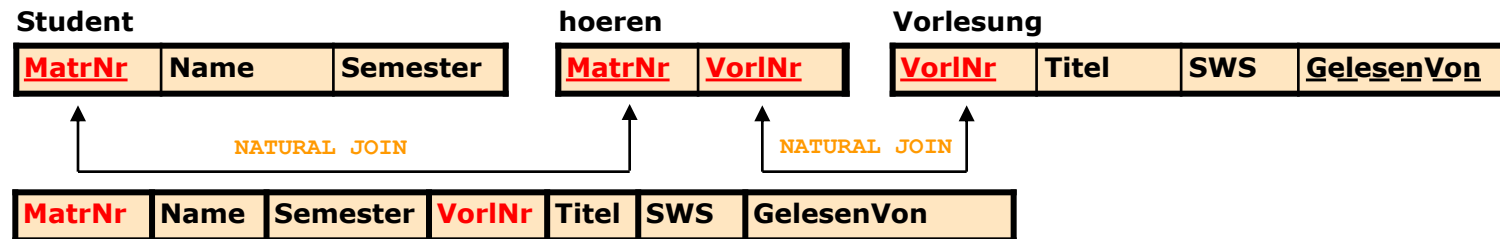
Anfragen über mehrere Relationen

Natürlicher Verbund – NATURAL JOIN

- Natürlicher Verbund als Abkürzung für ausführliche Anfrage

Welche Studierende besuchen welche Vorlesungen?

```
SELECT *
FROM Student NATURAL JOIN hoeren NATURAL JOIN Vorlesung;
```



- **NATURAL JOIN**-Klausel basiert auf allen Spalten, die in beiden Tabellen denselben Namen haben
 - Zeilen aus den beiden Tabellen werden ausgewählt, die in allen übereinstimmenden Spalten die **gleichen Werte** haben
 - Spalten mit denselben Namen, jedoch **unterschiedlichen Datentyp** verursachen einen **Syntax-Fehler**
 - Haben die Tabellen keine Spalten mit gleichem Namen, wird der **NATURAL JOIN** automatisch zum **CROSS JOIN**.

Anfragen über mehrere Relationen

(Natürlicher) Verbund – INNER JOIN und USING

- Verbund mit **USING**-Klausel
 - Mit der **USING**-Klausel kann angegeben werden, dass nur bestimmte Spalten verwendet werden sollen, wenn mehrere übereinstimmende Spalten existieren.

Beispiel

```
SELECT MatrNr, Name, Titel
FROM Student INNER JOIN hoeren USING (MatrNr)
INNER JOIN Vorlesung USING (VorlNr);
```

Anmerkung: **Verwendung eines allgemeinen Verbunds**

Anfragen über mehrere Relationen

Aliasnamen für Relationen

- Aliasnamen bzw. Tupelvariablen für Relationen erlauben
 - **mehrfachen Zugriff auf eine Relation** (**Selbstverbund**) bzw.
 - Zugriff auf **Attribute** mit **gleichen Namen** in **verschiedenen Relationen**:

Beispiel (Best Practice)

```
SELECT s.MatrNr, s.Name, v.Titel
FROM Student s INNER JOIN hoeren h ON (s.MatrNr = h.MatrNr)
      INNER JOIN Vorlesung v ON (h.VorlNr = v.VorlNr);
```

Ohne Aliasnamen

```
SELECT Student.MatrNr, Student.Name, Vorlesung.Titel
FROM Student INNER JOIN hoeren ON(Student.MatrNr = hoeren.MatrNr)
      INNER JOIN Vorlesung ON(hoeren.VorlNr = Vorlesung.VorlNr);
```

Anfragen über mehrere Relationen

Selbstverbund

■ Selbstverbund mit Aliasnamen

Beispiel

Welches sind die Vorlesungsnummern der Vorlesungen, die indirekte Vorgänger 2. Stufe der Vorlesung mit dem Titel 'Der Wiener Kreis' sind (= Vorgänger der Vorgänger von 'Der Wiener Kreis')?

Ergebnis

Vorgaenger
5043
5041

voraussetzen

Vorgaenger	Nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

'Der Wiener Kreis'

```
SELECT v1.vorgaenger
FROM voraussetzen v1 INNER JOIN voraussetzen v2
ON (v1.nachfolger = v2.vorgaenger) INNER JOIN Vorlesung v
ON (v2.nachfolger = v.VorlNr AND v.Titel = 'Der Wiener Kreis');
```

Anfragen über mehrere Relationen

EMPLOYEE

<u>Employee_Id</u>	Name	...	<u>Department_Id</u>
...	...	...	...

DEPARTMENT

<u>Department_Id</u>	Name	...	...
...	...	...	...

Anfragen über mehrere Relationen

EMPLOYEE

<u>Employee_Id</u>	Name	...	<u>Department_Id</u>
...	...	...	...

DEPARTMENT

<u>Department_Id</u>	Name	...	...
...	...	...	...

SELECT *

FROM EMPLOYEE NATURAL JOIN DEPARTMENT;

↳ gleich benannte Spalten werden nur einmal aufgenommen (Department_Id, Name)

Anfragen über mehrere Relationen

EMPLOYEE

<u>Employee_Id</u>	Name	...	<u>Department_Id</u>
...	...	...	...

DEPARTMENT

<u>Department_Id</u>	Name	...	...
...	...	...	...

SELECT *

FROM EMPLOYEE NATURAL JOIN DEPARTMENT;

↳ Gleich benannte Spalten werden nur einmal aufgenommen (Department_Id, Name)

FROM EMPLOYEE INNER JOIN DEPARTMENT USING (Department_Id);

↳ Department_Id wird nur einmal aufgenommen

↳ Name wird automatisch umbenannt und zweimal aufgenommen (Name, Name_1)

Anfragen über mehrere Relationen

EMPLOYEE

<u>Employee_Id</u>	Name	...	<u>Department_Id</u>
...	...	...	...

DEPARTMENT

<u>Department_Id</u>	Name	...	...
...	...	...	...

SELECT *

FROM EMPLOYEE NATURAL JOIN DEPARTMENT;

↳ Gleich benannte Spalten werden nur einmal aufgenommen (Department_Id, Name)

FROM EMPLOYEE INNER JOIN DEPARTMENT USING(Department_Id);

↳ Department_Id wird nur einmal aufgenommen

↳ Name wird automatisch umbenannt und zweimal aufgenommen (Name, Name_1)

SELECT e.Name

FROM EMPLOYEE e INNER JOIN DEPARTMENT d
ON (e.Department_Id = d.Department_Id)

Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte

Mengenoperationen

UNION, INTERSECT, EXCEPT

- Mengenoperatoren **eliminieren Duplikate**
- Mengenoperationen erfordern **typkompatible Wertebereiche** für Paare korrespondierender Attribute:
 - beide Wertebereiche sind **gleich** oder
 - beide sind auf **CHARACTER** basierende Wertebereiche (unabhängig von der Länge der Strings) oder
 - beide sind **numerische** Wertebereiche (unabhängig von dem genauen Typ) wie **INTEGER** oder **FLOAT**

Beispiel

```
SELECT A, B, C
FROM R1
UNION
SELECT A, C, D
FROM R2
```

- **Attributkompatibilität:**
A von R1 und A von R2,
B von R1 und C von R2,
C von R1 und D von R2
- **Ergebnis:**
Attributnamen des linken Operanden;
Resultatspalten bekommen jeweils
den allgemeineren Typ

Mengenoperationen

UNION

R1

A	B	C
1	2	3
2	3	4

R2

A	C	D
2	3	4
2	4	5

R1 UNION R2

A	B	C
1	2	3
2	3	4
2	4	5

Beispiel: UNION

Suchen Sie die Personalnummern und Namen aller Angestellten, also die aller Assistenten und aller Professoren.

```
SELECT PersNr, Name FROM Assistent
UNION
SELECT PersNr, Name FROM Professor;
```

Assistent

PersNr	Name	Fachgebiet	Boss
3002	Platon	Ideenlehre	2125
3003	Aristoteles	Syllogistik	2125
3004	Wittgenstein	Sprachtheorie	2126
3005	Rhetikus	Planetenbewegung	2127
3006	Newton	Kepler Gesetze	2127
3007	Spinoza	Gott und Natur	2126

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7



Ergebnis

PersNr	Name
2125	Sokrates
2126	Russel
2127	Kopernikus
2133	Popper
2134	Augustinus
2136	Curie
2137	Kant
3002	Platon
3003	Aristoteles
3004	Wittgenstein
3005	Rhetikus
3006	Newton
3007	Spinoza

Mengenoperationen

EXCEPT

Anmerkung: In Oracle
Schlüsselwort **MINUS** !

Beispiel: EXCEPT

Gesucht sind die Matrikelnummern der Studierenden, die noch keine Prüfung absolviert haben.

```
SELECT MatrNr FROM Student
EXCEPT
SELECT MatrNr FROM pruefen;
```

Student

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

pruefen

MatrNr	VorlNr	PersNr	Note
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2



Ergebnis

MatrNr
24002
26120
26830
29120
29555

Mengenoperationen

INTERSECT

Beispiel: INTERSECT

Gesucht sind die Personalnummern jener C4-Professoren, die mindestens eine Vorlesung halten?

```
SELECT GelesenVon FROM Vorlesung
INTERSECT
SELECT PersNr FROM Professor
WHERE Rang = 'C4';
```

Vorlesung

<u>VorlNr</u>	<u>Titel</u>	<u>SWS</u>	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Professor

<u>PersNr</u>	<u>Name</u>	<u>Rang</u>	<u>Raum</u>
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7



Ergebnis

<u>PersNr</u>
2137
2125
2126

Mengenoperationen

Duplikate

Anmerkung: In Oracle **MINUS** **ALL** und **INTERSECT** **ALL** nicht realisiert!

- **Duplikate** der an der Mengenoperation beteiligten Tabellen werden **erhalten**, sofern die Varianten

- UNION **ALL**
- INTERSECT **ALL** und
- EXCEPT **ALL**

verwendet werden (Standard ist **DISTINCT**).

■ Verhalten:

- Wenn erster Operand n Duplikate und zweiter Operand m Duplikate einer Zeile besitzt ($0 \leq n, m$), dann hat Ergebnis bei
 - UNION ALL $n+m$
 - INTERSECT ALL $\min(n, m)$
 - EXCEPT ALL $(n-m, 0)$

Duplikate dieses Tuples

SQL-1999	Oracle	DB2	Postgres	MySQL
UNION	✓	✓	✓	✓
UNION ALL	✓	✓	✓	✓
INTERSECT	✓	✓	✓	–
INTERSECT ALL	–	✓	✓	–
EXCEPT	(✓) MINUS	✓	✓	–
EXCEPT ALL	–	✓	✓	–

Überblick

■ Einführung Datenabfragesprache SQL

■ Datenabfragesprache SQL

- SQL-Kern
- Einfache SQL-Anfragen
- Anfragen über mehrere Relationen
- Mengenoperationen
- Aggregatfunktionen
- Aggregate und Gruppierung
- Geschachtelte Anfragen
- Existenziell quantifizierte Anfragen
- Allquantifizierte Anfragen
- Nullwerte
- Spezielle Sprachkonstrukte

1. Teil

Aggregatfunktionen

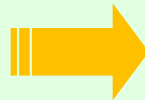
- Aggregatfunktionen sind Operationen,
 - die aus einer Menge von Tupeln **einen Wert** berechnen, z.B.
 - Summe über alle Werte einer Spalte
 - Durchschnitt über alle Werte einer Spalte

Beispiel:

Bestimmen Sie die durchschnittliche
Inskriptionsdauer der Studierenden?

Student

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2



Ergebnis

AVG(Semester)
7,625

Aggregatfunktionen

Standardfunktionen

■ In der **SELECT**-Klausel

- **COUNT**
 - berechnet **Anzahl der Werte** einer Spalte oder
 - im Spezialfall `COUNT (*)` **Anzahl der Tupel**
 - **SUM**
 - berechnet die Summe der Werte einer Spalte
 - nur bei numerischen Wertebereichen
 - **AVG**
 - berechnet den arithmetischen Mittelwert der Werte einer Spalte
 - nur bei numerischen Wertebereichen
 - **MAX** bzw. **MIN**
 - berechnen den größten bzw. kleinsten Wert einer Spalte
- Ignorieren **NULL**-Werte
- außer **COUNT (*)**
- Erzeugen genau einen Ergebnis-Tupel
- bei leerer Relation wird NULL-Wert zurückgeliefert

Oracle Aggregatfunktionen:

AVG
 COLLECT
 CORR
 CORR_*
 COUNT
 COVAR_POP
 COVAR_SAMP
 CUME_DIST
 DENSE_RANK
 FIRST
 GROUP_ID
 GROUPING
 GROUPING_ID
 LAST
 MAX
 MEDIAN
 MIN
 PERCENTILE_CONT
 PERCENTILE_DISC
 PERCENT_RANK
 RANK
 REGR_ (Linear Regression) Functions
 STATS_BINOMIAL_TEST
 STATS_CROSSTAB
 STATS_F_TEST
 STATS_KS_TEST
 STATS_MODE
 STATS_MW_TEST
 STATS_ONE_WAY_ANOVA
 STATS_T_TEST_*
 STATS_WSR_TEST
 STDDEV
 STDDEV_POP
 STDDEV_SAMP
 SUM
 VAR_POP
 VAR_SAMP
 VARIANCE

Aggregatfunktionen

Argumente

- **Argumente** einer Aggregatfunktion
 - ein Attribut der durch die **FROM**-Klausel spezifizierten Relation
 - im Falle der **COUNT**-Funktion auch das Symbol *****
- **Vor dem Argument** (außer bei **COUNT (*)**) **optional**
 - **DISTINCT**
 - bewirkt, dass die Funktion keine doppelten Werte berücksichtigt.
 - **ALL**
 - bewirkt, dass alle Werte, einschließlich der doppelten Werte, berücksichtigt werden (**Default**).

Aggregatfunktionen

Argumente

- Eine **Duplikatelimination** kann **erzwungen** werden
 - **COUNT (DISTINCT GelesenVon)** zählt verschiedene Professoren
 - **COUNT (ALL GelesenVon)** zählt alle Einträge (außer NULL)
 - **COUNT (GelesenVon)** ist identisch mit **COUNT (ALL GelesenVon)**
 - **COUNT (*)** zählt die Tupel des Anfrageergebnisses

<i>R</i>	<i>select *</i> <i>from R</i>	<i>select count(*)</i> <i>from R</i>	<i>select count(A)</i> <i>from R</i>	<i>select count(DISTINCT A)</i> <i>from R</i>
A	A	count	count	count
3	3	3	2	1
3	3			
null	null			

Aggregatfunktionen

Nullwerte

- Aggregatfunktionen:
 - Ignorieren Nullwerte in den aggregierten Attributen
 - Ausnahme: **COUNT (*)** ermittelt die Zeilenanzahl einer Relation

Beispiel:

```
SELECT COUNT (SWS)
FROM Vorlesung;
```

Vorlesung

<u>VorlNr</u>	<u>Titel</u>	<u>SWS</u>	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	NULL	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	NULL	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

- Anfrage zählt keine Vorlesungen mit einem Nullwert als SWS
- Das Resultat ist 0, falls alle SWS-Einträge **NULL** sind

Aggregatfunktionen

Beispiel:

Bestimmen Sie die durchschnittliche/maximale/minimale Inskriptionsdauer der Studierenden?

Bestimmen Sie die Anzahl der Studierenden?

Student

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

```
SELECT AVG (Semester) FROM Student;
```

```
SELECT MAX (Semester) FROM Student;
```

```
SELECT MIN (Semester) FROM Student;
```

```
SELECT COUNT (MatrNr) FROM Student;
```

AVG(Semester)

7,625

MAX(Semester)

18

MIN(Semester)

2

COUNT(MatNr)

8

Aggregatfunktionen



Beispiel:

Geben Sie die Summe der von C4-ProfessorInnen gehaltenen Vorlesungseinheiten an?

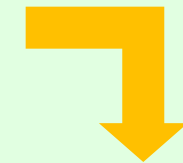
```
SELECT SUM(SWS) AS "C4-Vorlesungseinheiten"
FROM Vorlesung INNER JOIN Professor ON (gelesenVon = PersNr)
WHERE Rang = 'C4';
```

Vorlesung

VorlNr	Titel	SWS	GelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7



C4-Vorlesungseinheiten
26

Aggregatfunktionen

... etwas Statistik!



Beispiel:

Ermitteln Sie die das arithmetische Mittel, den Median, die Standardabweichung sowie die Varianz für die Körpergröße der Studierenden?

```
SELECT  AVG(Groesse) AS "Arithmetisches Mittel",
        MEDIAN(Groesse) AS "Median",
        ROUND(STDDEV(Groesse),2) AS "Standardabweichung",
        ROUND(VARIANCE(Groesse),2) AS "Varianz/Streuung"
FROM Student;
```

Student

MatrNr	Name	Semester	Groesse
24002	Xenokrates	18	175
25403	Jonas	12	195
26120	Fichte	10	162
26830	Aristoxenos	8	174
27550	Schopenhauer	6	189
28106	Carnap	3	178
29120	Theophrastos	2	165
29555	Feuerbach	2	170



Ergebnis

Arithmetisches Mittel	Median	Standardabweichung	Varianz/Streuung
176	174,5	11,29	127,43

Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte

Aggregate und Gruppierung

ID	Value
1	50.30
1	123.30
1	132.90
2	50.30
2	123.30
2	132.90
2	88.90
3	50.30
3	123.30

ID	Value
1	306.50
2	395.40
3	173.60

■ Problem

- Wollen nicht die Summe aller Vorlesungsstunden absolut berechnen, sondern zu jedem/r Vortragenden die Summe der von ihm/ihr gehaltenen Stunden.

■ Lösung

- Bilde zu jedem/r Vortragenden eine Gruppe (mit der **GROUP BY**-Klausel) und summiere nur innerhalb dieser Gruppe.

■ Allgemein

- Alle Zeilen einer Tabelle, die auf den Attributen der **GROUP BY**-Klausel den selben Wert annehmen, werden zu einer Gruppe zusammengefasst und die Aggregatfunktionen werden bezüglich der Gruppe ausgewertet.

■ Bedingungen

- an die aggregierten Werte werden in der **HAVING**-Klausel angeführt.

Gruppierung 1/6

GROUP BY- und HAVING-Klauseln

Notation:

SELECT ...

FROM ...

[**WHERE** bedingungen] —————> **Auswahl Tabellenzeilen**

[**GROUP BY** attributliste] —————> **Gruppierungen von Tabellenzeilen**

[**HAVING** bedingungen] ; —————> **Auswahl Gruppierungen**

- **GROUP BY**-Operator unterteilt die in der **FROM**-Klausel angegebene Tabelle in Gruppen, so dass alle Tupel innerhalb einer Gruppe den gleichen Wert für die **GROUP BY**-Attribute haben.
- Dabei werden alle Tupel, die die **WHERE**-Klausel nicht erfüllen, eliminiert.
- Danach werden innerhalb jeder solchen Gruppe die Aggregatfunktionen berechnet.
- Die **HAVING**-Klausel ermöglicht es, Bedingungen an die durch **GROUP BY** gebildeten Gruppen zu formulieren.
- In der **SELECT**-Klausel dürfen nur zwei Spaltenarten vorkommen
 - die, die mit einer Aggregatfunktion versehen sind und
 - die anderen müssen in der **GROUP BY**-Klausel enthalten sein.

SQL-Kern

Auswertungsreihenfolge einen SFW-Blocks

Auswertungsreihenfolge

- ⑥ **SELECT**
 - Projektionsliste (= Ergebnisschema)
 - arithmetische Operationen und Aggregatfunktionen
- ① **FROM**
 - zu verarbeitende Relationen, evtl. Umbenennungen
- ② **WHERE**
 - Selektions-, Verbundbedingungen
 - Geschachtelte Anfragen (wieder ein SFW-Block)
- ③ **GROUP BY**
 - Gruppierung für Aggregatfunktionen
- ④ **HAVING**
 - Selektionsbedingungen an Gruppen
- ⑤ **ORDER BY**
 - Sortierbedingungen an Gruppen

Aggregate und Gruppierung 2/6

GROUP BY- und HAVING-Klauseln

REL

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	6	8



ERGEBNIS

A	SUM(D)
1	9

Anfrage:

```

SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY A, B
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;

```

Aggregate und Gruppierung 3/6

GROUP BY- und HAVING-Klauseln

■ Schritt 1: FROM und WHERE

REL

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	6	8



WHERE-Klausel:
Bedingungen an einzelne
Tupel bevor gruppiert wird

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7

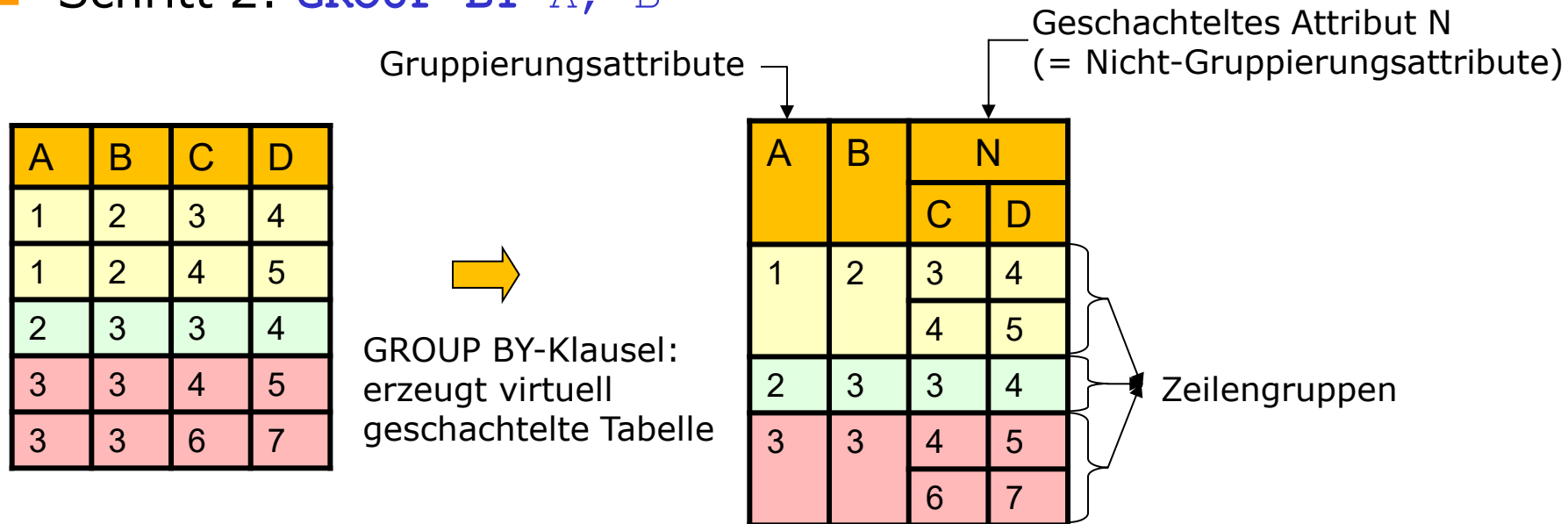
Anfrage:

```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY A, B
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```


Aggregate und Gruppierung 4/6

GROUP BY- und HAVING-Klauseln

■ Schritt 2: GROUP BY A, B



Anfrage:

```

SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY A, B
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;

```

Aggregate und Gruppierung 5/6

GROUP BY- und HAVING-Klauseln

- Schritt 3: **HAVING** $A < 4$ **AND** **SUM**(D) < 10 **AND** **MAX**(C) $= 4$

A	B	N	
		C	D
1	2	3	4
		4	5
2	3	3	4
3	3	4	5
		6	7

Aggregation

A	B	N	
		SUM(D)	MAX(C)
1	2	9	4
2	3	4	3
3	3	12	6

HAVING-Klausel
(=Selektion)

A	B	N	
		SUM(D)	MAX(C)
1	2	9	4

Anfrage:

```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY A, B
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```

In der **HAVING**-Klausel dürfen neben Aggregatfunktionen nur Attribute vorkommen, die explizit in der **GROUP BY**-Klausel aufgeführt wurden.

Aggregate und Gruppierung 6/6

GROUP BY- und HAVING-Klauseln

■ Schritt 4: **SELECT** A, **SUM**(D)

A	B	N	
		SUM(D)	MAX(C)
1	2	9	4



SELECT-Klausel
(=Projektion)

A	SUM(D)
1	9

Anfrage:

```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY A, B
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```

Aggregate und Gruppierung

Beispiel:

Geben Sie zu jedem/r C4-Professor*in die Summe der von ihm/ihr gehaltenen Vorlesungsstunden an?

```
SELECT gelesenVon, SUM(SWS)
FROM Vorlesung INNER JOIN Professor
           ON (gelesenVon = PersNr)
WHERE Rang = 'C4'
GROUP BY gelesenVon;
```

Ergebnis

gelesenVon	SUM(SWS)
2125	10
2126	8
2137	8

Aggregate und Gruppierung

Beispiel:

Geben Sie zu jedem/r C4-Professor*in, der/die vor allem lange Vorlesungen (durchschnittlich größer gleich 3 SWS) hält, die Summe der von ihm/ihr gehaltenen Vorlesungsstunden an?

```
SELECT gelesenVon, Name, SUM(SWS)
FROM Vorlesung INNER JOIN Professor
      ON gelesenVon = PersNr
WHERE Rang = 'C4'
GROUP BY gelesenVon, Name
HAVING AVG(SWS) >= 3;
```

Ergebnis

gelesenVon	Name	SUM(SWS)
2125	Sokrates	10
2137	Kant	8

Aggregate und Gruppierung

Abarbeitung

FROM Vorlesung **INNER JOIN** Professor **ON** gelesenVon = PersNr



<i>Professor ⋈_{PersNr=gelesenVon} Vorlesung</i>							
PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon
2125	Sokrates	C4	226	5001	Grundzüge	4	2137
2125	Sokrates	C4	226	5041	Ethik	4	2125
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
2125	Sokrates	C4	226	5049	Mäeutik	2	2125
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
2133	Popper	C3	52	5001	Grundzüge	4	2137
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
2137	Kant	C4	7	4630	Die drei Kritiken	4	2137

WHERE Rang = 'C4'



Aggregate und Gruppierung

Abarbeitung

WHERE Rang = 'C4'



PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon
2137	Kant	C4	7	5001	Grundzüge	4	2137
2125	Sokrates	C4	226	5041	Ethik	4	2125
2126	Russel	C4	232	5043	Erkenntnistheorie	3	2126
2125	Sokrates	C4	226	5049	Mäeutik	2	2125
2125	Sokrates	C4	226	4052	Logik	4	2125
2126	Russel	C4	232	5052	Wissenschaftstheorie	3	2126
2126	Russel	C4	232	5216	Bioethik	2	2126
2137	Kant	C4	7	4630	Die drei Kritiken	4	2137

GROUP BY gelesenVon, Name



Aggregate und Gruppierung

Abarbeitung

GROUP BY gelesenVon, Name



PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon	
2125	Sokrates	C4	226	5041	Ethik	4	2125	AVG=3,3
2125	Sokrates	C4	226	5049	Mäeutik	2	2125	
2125	Sokrates	C4	226	4052	Logik	4	2125	
2126	Russel	C4	232	5043	Erkenntnistheorie	3	2126	AVG=2,3
2126	Russel	C4	232	5052	Wissenschaftstheorie	3	2126	
2126	Russel	C4	232	5216	Bioethik	2	2126	
2137	Kant	C4	7	5001	Grundzüge	4	2137	AVG=4
2137	Kant	C4	7	4630	Die drei Kritiken	4	2137	

HAVING AVG (SWS) >= 3;



Aggregate und Gruppierung

Abarbeitung

HAVING AVG (SWS) >= 3



PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon
2125	Sokrates	C4	226	5041	Ethik	4	2125
2125	Sokrates	C4	226	5049	Mäeutik	2	2125
2125	Sokrates	C4	226	4052	Logik	4	2125
2137	Kant	C4	7	5001	Grundzüge	4	2137
2137	Kant	C4	7	4630	Die drei Kritiken	4	2137

AVG=3,3 AVG=4

SELECT gelesenVon, Name, **SUM**(SWS)



gelesenVon	Name	sum(SWS)
2125	Sokrates	10
2137	Kant	8

Aggregate und Gruppierung

■ Achtung

- SQL erzeugt pro Gruppe ein Ergebnistupel
- Alle in der **SELECT**-Klausel aufgeführten Attribute - außer den aggregierten - müssen auch in der **GROUP BY**-Klausel aufgeführt werden. So wird sichergestellt, dass die ausgegebenen Attribute sich nicht innerhalb der Gruppe ändern.
- Daher muss hier auch das Attribut „Name“ in der **GROUP BY**-Klausel auch aufgenommen werden, obwohl es eigentlich nicht notwendig wäre, da gelesenVon immer Name impliziert. Ist aber notwendig, da man in der **SELECT**-Klausel auf dieses Attribut zugreifen möchte.

```
SELECT gelesenVon, Name, SUM(SWS)
...
GROUP BY gelesenVon, Name;
```

Aggregate und Gruppierung

 $\mathbb{R}$

A	B			Z

SELECT A, B, SUM(Z)

FROM R

GROUP BY A, B

Aggregate und Gruppierung

R

A	B			Z

SELECT A, B, SUM(Z)
 FROM R
 GROUP BY A, B

es können nur
 Gruppierungs- und
 Aggregationsattribute
 aufscheinen

Aggregate und Gruppierung

$\mathbb{R}$

A	B	C		Z

SELECT A, B, SUM(Z)
 FROM R
 GROUP BY A, B

es können nur
 Gruppierungs- und
 Aggregationsattribute
 aufscheinen

Aggregate und Gruppierung

R

A	B	C		Z

SELECT

A, B, C, SUM(Z)

es können nur
Gruppierungs- und
Aggregationsattribute
aufscheinen

FROM R

GROUP BY

A, B, C

es entstehen dadurch
mehrere Gruppen

↳ DRILL DOWN

vs.

ROLL UP

Gruppierung

- **GROUP BY-Klausel** bewirkt Teilmengenbildung (Gruppierung) der Ergebnis-Relation entsprechend Gruppierungsattribut(en).
- Man kann die **WHERE-Klausel nicht** verwenden, um **Gruppen einzuschränken**.
- Man verwendet die **HAVING-Klausel**, um **Gruppen einzuschränken**. **HAVING** kann nur in Verbindung mit **GROUP BY** auftreten.
- Aggregatfunktionen werden auf Teilmengen angewandt.
- Als **Aggregatfunktionen** dürfen in der **HAVING-Klausel** **MIN**, **MAX**, **COUNT**, **SUM** und **AVG** verwendet werden.
- Man kann **keine Aggregatfunktionen** in der **WHERE-Klausel** verwenden
 - außer in der **WHERE-Klausel** ist wieder ein SFW-Block mit Aggregatfunktion enthalten
- Verbund wird vor der Gruppierung ausgeführt.

Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte

Wozu geschachtelte Anfragen?

■ Die Anfrage:

- Wie heißen die Professor*innen mit dem geringsten Gehalt in unserer Hochschule?

■ Das Problem:

- Wir müssten erst einmal wissen, wie hoch das geringste Gehalt ist, um anschließend zu ermitteln, welche Professor*innen dieses Gehalt beziehen.

■ Die Lösung:

1. Ermittle zuerst in der geschachtelten Anfrage das niedrigste Gehalt über alle Professor*innen.
2. Nimm den Betrag des niedrigsten Gehaltes und ermittle in einer Anfrage alle Professor*innen, die ein Gehalt in dieser Höhe beziehen.
3. Gib die ermittelten Professor*innen aus.

Geschachtelte Anfragen

- Da das **Ergebnis** von **jeder SQL-Anfrage** immer eine **Tabelle** ist, können **SQL-Anfragen grundsätzlich verschachtelt** werden („**Abgeschlossenheit**“)
- **Unterabfragen** können als **Zwischenschritt** eingesetzt werden, um **komplexe Anfragen** in einer **einzelnen Anfrage** zu beantworten
- Je nachdem, welches **konkrete Ergebnis** man von einer Unterabfrage erwartet, kann es an unterschiedlichen Stellen in der SQL-Anfrage verwendet werden
 - Wenn eine Unterabfrage als Ergebnis einen **einzelnen Wert** (d.h., ein Tupel mit einem Attribut) liefert, kann sie in **SELECT**- oder **WHERE**-Klausel verwendet werden
 - Wenn eine Unterabfrage als Ergebnis eine **Liste von Werten** (d.h., eine Tabelle mit einer Spalte) liefert, kann sie bei **IN**, **NOT IN**, **ANY**, **ALL** verwendet werden
 - Wenn eine Unterabfrage eine **Tabelle** liefert, kann sie in **FROM**-Klausel und mit **EXISTS** verwendet werden

Anmerkungen:

- Bei geschachtelten Anfragen ist für die Laufzeit ausschlaggebend, ob es sich um **korrelierte** oder **nicht korrelierte** Anfragen handelt
- Unterabfragen müssen **immer geklammert** werden, egal, wo sie stehen
- Teilweise können **Unterabfragen** durch „**normale**“ **Anfragen** formuliert werden (**entschachtelt**)

Geschachtelte Anfragen

- Unteranfrage liefert **einen Wert**; typischerweise berechnen solche Unteranfragen eine Aggregatfunktion
- Beispiel für Verwendung in der **WHERE-Klausel**

Beispiel:

Welche Studierende studieren am längsten?

```
SELECT Name
FROM Student
WHERE Semester = ( SELECT MAX (Semester)
                   FROM Student );
```

Operator kann nur **skalaren Vergleich** durchführen
→ keine Mengenvergleiche möglich!

} ein Tupel mit einem Attribut

Student

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2



Ergebnis

Name
Xenokrates

Nicht korrelierte Unterabfrage
→ wird nur **einmal ausgeführt**!

Geschachtelte Anfragen

Korrelierte Unterabfrage (greift auf Attribute der umschließenden Anfrage zu) → Unterabfrage wird für **jedes Ergebnistupel einmal ausgeführt!**
→ Performance!!!

- Unteranfrage liefert **einen Wert**
- Beispiel für Verwendung in der **SELECT-Klausel**

Beispiel:

Geben Sie zu jedem Professor seine Lehrbelastung aus.

```
SELECT p.PersNr, p.Name, ( SELECT SUM(v.SWS)
                           FROM Vorlesung v
                           WHERE v.gelesenVon = p.PersNr )
FROM Professor p;
```

ein Tupel mit einem Attribut

korreliert

Vorlesung

VorlNr	Titel	SWS	GelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Ergebnis

PersNr	Name	Lehrbelastung
2125	Sokrates	10
2126	Russel	8
2127	Kopernikus	null
2133	Popper	2
2134	Augustinus	2
2136	Curie	null
2137	Kant	8

Alternative Formulierung → Entschachtelt



- Es ist **besser** und **effizienter** es folgendermaßen zu formulieren:

Beispiel:

Geben Sie zu jedem Professor seine Lehrbelastung aus.

```
SELECT p.PersNr, p.Name, SUM(v.SWS) AS Lehrbelastung
FROM Vorlesung v INNER JOIN Professor p ON (v.gelesenVon = p.PersNr)
GROUP BY p.PersNr, p.Name;
```

Ist diese Abfrage zu 100% äquivalent zu vorhergehenden Abfrage?

Alternative Formulierung → Entschachtelt



- Es ist **besser** und **effizienter** es folgendermaßen zu formulieren:

Beispiel:

Geben Sie zu jedem Professor seine Lehrbelastung aus.

```
SELECT p.PersNr, p.Name, SUM(v.SWS) AS Lehrbelastung
FROM Vorlesung v INNER JOIN Professor p ON (v.gelesenVon = p.PersNr)
GROUP BY p.PersNr, p.Name;
```

Ist diese Abfrage zu 100% äquivalent zu vorhergehenden Abfrage?

```
-- Hier erscheinen allerdings jene Professoren nicht, die keine
-- Lehrbelastung haben
```

```
-- Äquivalente Alternative mit Outer Join
```

```
SELECT p.PersNr, p.Name, SUM(v.SWS) AS Lehrbelastung
FROM Vorlesung v RIGHT OUTER JOIN Professor p ON (v.gelesenVon =
p.PersNr)
GROUP BY p.PersNr, p.Name;
```

Geschachtelte Anfragen

- Da das **Ergebnis** von **jeder SQL-Anfrage** immer eine **Tabelle** ist, können **SQL-Anfragen grund-sätzlich verschachtelt** werden („**Abgeschlossenheit**“)
- **Unterabfragen** können als **Zwischenschritt** eingesetzt werden, um **komplexe Fragen** in einer **einzelnen Anfrage** zu beantworten
- Je nachdem, welches **konkrete Ergebnis** man von einer Unterabfrage erwartet, kann es an unterschiedlichen Stellen in der SQL-Anfrage verwendet werden
 - Wenn eine Unterabfrage als Ergebnis einen **einzelnen Wert** (d.h., ein Tupel mit einem Attribut) liefert, kann sie in **SELECT**- oder **WHERE**-Klausel verwendet werden
 - Wenn eine Unterabfrage als Ergebnis eine **Liste von Werten** (d.h., eine Tabelle mit einer Spalte) liefert, kann sie bei **IN, NOT IN, ANY, ALL** verwendet werden
 - Wenn eine Unterabfrage eine **Tabelle** liefert, kann sie in **FROM**-Klausel und mit **EXISTS** verwendet werden

Anmerkungen:

- Bei geschachtelten Anfragen ist für die Laufzeit ausschlaggebend, ob es sich um **korrelierte** oder **nicht korrelierte** Anfragen handelt
- Unterabfragen müssen **immer geklammert** werden, egal, wo sie stehen
- Teilweise können **Unterabfragen** durch „normale“ **Anfragen** formuliert werden (**entschachtelt**)

Geschachtelte Anfragen

Mengenvergleich mit IN-Operator

implizite Mengendefinition (= Unteranfrage berechnet Menge **atomarer Werte**)

explizite Mengendefinition (Konstante)

Notation:

Attribut **IN** (SFW-Block | Aufzählung)

Beispiel:

Suchen Sie alle jene Professoren, die Lehrveranstaltungen halten.

```
SELECT Name
FROM Professor
WHERE PersNr IN (SELECT gelesenVon FROM Vorlesung);
```

Vorlesung

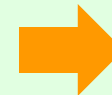
<u>VorlNr</u>	Titel	SWS	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Professor

<u>PersNr</u>	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Ergebnis

Name
Sokrates
Russel
Popper
Augustinus
Kant



Geschachtelte Anfragen

Mengenvergleich mit IN-Operator

Äußere SELECT-Anweisung

```
SELECT Name
FROM Professor
WHERE PersNr IN (SELECT gelesenVon FROM Vorlesung);
```

Innere SELECT-Anweisung

■ Abarbeitung

1. Ergebnis der inneren **SELECT**-Anweisung ermitteln
2. In explizite Form (2125,2126,2133,2134,2137) umwandeln
3. Hinter **IN** als Liste von Konstanten einsetzen

```
SELECT Name
FROM Professor
WHERE PersNr IN (2125,2126,2133,2134,2137);
```

- Modifizierte äußere **SELECT**-Anweisung auswerten
- Erscheint **PersNr** in dieser Liste auf (=Test auf Mengenzugehörigkeit, so wird **TRUE** zurückgeliefert und der zugehörige Professor in das Ergebnis übernommen

Geschachtelte Anfragen

Negation des IN-Operators

Beispiel:

Suchen Sie alle jene Professoren, die keine Lehrveranstaltungen halten.

```
SELECT Name
FROM Professor
WHERE PersNr NOT IN (SELECT gelesenVon FROM Vorlesung);
```

Vorlesung

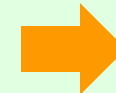
VorlNr	Titel	SWS	GelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Professor

PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Ergebnis

Name
Kopernikus
Curie



Geschachtelte Anfragen

Mengenvergleich mit IN-Operator

Beispiel:

Suchen Sie die Matrikelnummern und die Vorlesungsnummern aller geprüften Studierenden, die die geprüften Vorlesungen auch gehört haben.

```
SELECT *
FROM hoeren
WHERE (MatrNr, VorlNr) IN (SELECT MatrNr, VorlNr FROM pruefen);
```

hoeren

<u>MatrNr</u>	<u>VorlNr</u>
26120	5001
27550	5001
27550	4052
28106	5041
28106	5001
28106	4052
28106	4630
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

pruefen

<u>MatrNr</u>	<u>VorlNr</u>	<u>PersNr</u>	<u>Note</u>
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

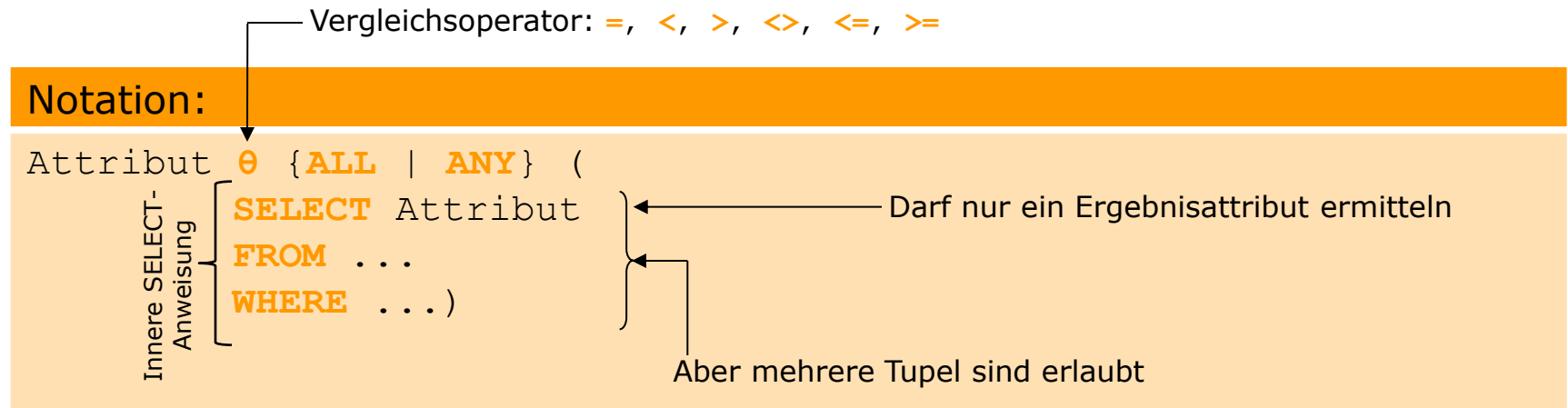


Ergebnis

<u>MatrNr</u>	<u>VorlNr</u>

Geschachtelte Anfragen

Mengenvergleich mit **ALL**- und **ANY**-Quantoren



- Vergleich eines Wertes mit einer Menge von Werten:
 - **θ ALL**: Bedingung wird zu **TRUE** ausgewertet, wenn der **θ**-Vergleich für alle Ergebniswerte der inneren **SELECT**-Anweisung **TRUE** ist
 - **θ ANY**: analog, wenn der **θ**-Vergleich für einen Ergebniswert **TRUE** ist

Geschachtelte Anfragen

Mengenvergleich mit **ALL**

$$(5 < \mathbf{all} \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{false}$$

$$(5 < \mathbf{all} \begin{array}{|c|} \hline 6 \\ \hline 10 \\ \hline \end{array}) = \text{true}$$

$$(5 = \mathbf{all} \begin{array}{|c|} \hline 4 \\ \hline 5 \\ \hline \end{array}) = \text{false}$$

Geschachtelte Anfragen

Mengenvergleich mit **ALL**

Beispiel:

Suchen Sie jene Studierenden, die am längsten studieren.

```
SELECT Name
FROM Student
WHERE Semester >= ALL
```

< **ALL** bedeutet weniger als das Minimum

> **ALL** bedeutet mehr als das Maximum

18, 12, 10, 8, ...

```
(
  SELECT Semester
  FROM Student
);
```

Liefert **SELECT** keinen Wert, so trifft die Auswahlbedingung **zu!**

Student

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2



Ergebnis

Name
Xenokrates

Geschachtelte Anfragen

Alternative Formulierung

Beispiel:

Suchen Sie jene Studierenden, die am längsten studieren.

```
SELECT Name
FROM Student
WHERE Semester = (SELECT MAX(Semester) FROM Student);
```



- Gibt es einen Unterschied zwischen **NOT IN (...)** und **<> ALL**?
 - NEIN!

Geschachtelte Anfragen

Mengenvergleich mit **ANY**

$(5 < \text{any } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline 6 \\ \hline \end{array}) = \text{true}$ (Bedeutung: $5 < \text{ein Tupel in der Tabelle}$)

$(5 < \text{any } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{false}$

$(5 = \text{any } \begin{array}{|c|} \hline 0 \\ \hline 5 \\ \hline \end{array}) = \text{true}$

Geschachtelte Anfragen

Mengenvergleich mit **ANY**

Beispiel:

Suchen Sie jene Vorlesungen, deren SWS-Anzahl geringer ist als die SWS-Anzahl irgendeiner Vorlesung von Professor Sokrates (`PersNr = 2125`).

```
SELECT Titel
FROM Vorlesung
WHERE gelesenVon <> 2125 AND
      SWS < ANY
```

< **ANY** bedeutet weniger als das Maximum
> **ANY** bedeutet mehr als das Minimum

```
      (
        SELECT SWS
        FROM Vorlesung
        WHERE gelesenVon = 2125
      ) ;
```

4, 2, 4

Liefert **SELECT** keinen Wert, so
trifft die Auswahlbedingung
nicht zu!

Geschachtelte Anfragen

- Da das **Ergebnis** von **jeder SQL-Anfrage** immer eine **Tabelle** ist, können **SQL-Anfragen grund-sätzlich verschachtelt** werden („**Abgeschlossenheit**“)
- **Unterabfragen** können als **Zwischenschritt** eingesetzt werden, um **komplexe Fragen** in einer **einzelnen Anfrage** zu beantworten
- Je nachdem, welches **konkrete Ergebnis** man von einer Unterabfrage erwartet, kann es an unterschiedlichen Stellen in der SQL-Anfrage verwendet werden

- Wenn eine Unterabfrage als Ergebnis einen **einzelnen Wert** (d.h., ein Tupel mit einem Attribut) liefert, kann sie in **SELECT**- oder **WHERE**-Klausel verwendet werden
- Wenn eine Unterabfrage als Ergebnis eine **Liste von Werten** (d.h., eine Tabelle mit einer Spalte) liefert, kann sie bei **IN**, **NOT IN**, **ANY**, **ALL** verwendet werden
- Wenn eine Unterabfrage eine **Tabelle** liefert, kann sie in **FROM**-Klausel und mit **EXISTS** verwendet werden

Anmerkungen:

- Bei geschachtelten Anfragen ist für die Laufzeit ausschlaggebend, ob es sich um **korrelierte** oder **nicht korrelierte** Anfragen handelt
- Unterabfragen müssen **immer geklammert** werden, egal, wo sie stehen
- Teilweise können **Unterabfragen** durch „normale“ **Anfragen** formuliert werden (**entschachtelt**)

Geschachtelte Anfragen

- Unteranfrage liefert **eine Tabelle**
- Beispiel für Verwendung in **FROM-Klausel**

Ohne Unterabfrage hätte man die Bedingung auf `VorlAnzahl > 2` nur mittels `HAVING` spezifizieren können.

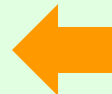
Beispiel:

Gesucht sind jene Studierenden, die mehr als zwei Vorlesungen hören.

```
SELECT tmp.MatrNr, tmp.Name, tmp.VorlAnzahl
FROM ( SELECT MatrNr, Name, COUNT(*) AS VorlAnzahl
      FROM Student NATURAL JOIN hoeren
      GROUP BY MatrNr, Name) tmp
WHERE tmp.VorlAnzahl > 2;
```

Ergebnis

MatrNr	Name	VorlAnzahl
28106	Carnap	4
29120	Theophrastos	3



Student

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

hoeren

MatrNr	VorlNr
26120	5001
27550	5001
27550	4052
28106	5041
28106	5001
28106	4052
28106	4630
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

Alternative Formulierung → Entschachtelt

- Entschachtelung der Unteranfrage in der **FROM**-Klausel mittels **HAVING**

Beispiel (Entschachtelung mittels **HAVING**):

Gesucht sind jene Studierenden, die mehr als zwei Vorlesungen hören.

```
SELECT MatrNr, Name, COUNT(*) AS VorlAnzahl
FROM Student NATURAL JOIN hoeren
GROUP BY MatrNr, Name
HAVING COUNT(*) > 2;
```

Ergebnis

MatrNr	Name	VorlAnzahl
28106	Carnap	4
29120	Theophrastos	3



Student

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

hoeren

MatrNr	VorlNr
26120	5001
27550	5001
27550	4052
28106	5041
28106	5001
28106	4052
28106	4630
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

Geschachtelte Anfragen

Existenzbedingung mit EXISTS-Operator

- Existenzquantor **EXISTS** prüft, ob Zeilen in der Ergebnismenge der Unteranfrage enthalten sind.

Beispiel:

Suchen Sie alle jene Studierenden, die älter als der/die jüngste ProfessorIn sind.

```

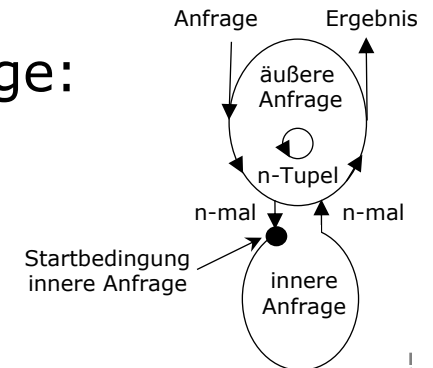
SELECT s.*
FROM Student s
WHERE EXISTS (
    SELECT p.*
    FROM Professor p
    WHERE p.GebDatum > s.GebDatum);
  
```

korreliert

Geschachtelte Abfrage wird für jeden Studierenden ausgeführt

- „Verzahnt“ geschachtelte (**korrelierte**) Anfrage:

- in der inneren Anfrage wird Tupelvariablen-Name aus dem **FROM**-Teil der äußeren Anfrage verwendet
- korrelierte Anfrage wird für jedes Tupel der umgebenden Anfrage neu berechnet



Geschachtelte Anfragen

Existenzbedingung mit EXISTS-Operator

Beispiel:

Suchen Sie alle jene Professoren, die keine Lehrveranstaltungen halten.

```
SELECT Name
FROM Professor p
WHERE NOT EXISTS ( SELECT *
                    FROM Vorlesung v
                    WHERE v.gelesenVon = p.PersNr);
```

$O(|p| * |v|) = n^2$

- Korrelierte Unteranfrage (PersNr und Professor)!
- Eine nicht korrelierte Lösung mit IN-Operator ist:

```
SELECT Name
FROM Professor
WHERE PersNr NOT IN ( SELECT gelesenVon
                     FROM Vorlesung);
```

Geschachtelte Anfragen

Existenzbedingung mit EXISTS-Operator

Beispiel:

Suchen Sie jene Assistent*innen, die für einen Professor*in arbeiten, der/die jünger ist, als sie selbst.

```
SELECT a.*
FROM Assistent a
WHERE EXISTS (
    SELECT p.*
    FROM Professor p
    WHERE a.Boss = p.PersNr AND
    p.GebDatum > a.GebDatum);
```

- Korrelierte Unteranfrage (a.GebDatum und Assistenten a)!
- Eine nicht korrelierte Lösung mit JOIN-Operator ist:

```
SELECT a.*
FROM Assistent a INNER JOIN Professor p ON a.Boss = p.PersNr
WHERE p.GebDatum > a.GebDatum;
```

Geschachtelte Anfragen

Allquantifizierende Anfragen

- Allquantifizierende Anfrage kann immer durch eine COUNT-Aggregation ausgedrückt werden.

Beispiel:

Suchen Sie all jene Studierende, die alle 4-stündigen Lehrveranstaltungen gehört haben.

1. Zähle zu jedem Studierenden, wie viele 4-stündigen LVAs er/sie besucht hat.
2. Zähle wie viele 4-stündigen LVAs existieren.

```
SELECT matrnr, name
FROM Student NATURAL JOIN hoeren NATURAL JOIN Vorlesung
WHERE sws = 4
GROUP BY matrnr, name
HAVING COUNT (*) = ( SELECT COUNT (*) FROM Vorlesung
                     WHERE sws = 4 );
```


Geschachtelte Anfragen

Zusammenfassung

- **Geschachtelte Anfragen** sind Anfragen mit Unteranfragen.
- Unteranfragen in der **WHERE**-Klausel können folgende Konstrukte verwenden:
 - **(NOT) IN**
 - **ANY**
 - **ALL**
 - **(NOT) EXISTS**
- Alle Unteranfragen können mit **(NOT) EXISTS** ausgedrückt werden.
- Eine **abgeleitete Tabelle** ist eine Unteranfrage in der **FROM**-Klausel.

Geschachtelte Anfragen

Zusammenfassung – Unterschied **IN**, **ALL**, **ANY**

- **IN** prüft, ob ein Wert in der Liste enthalten ist.

- [...] 5 IN (1,2,3,4,5) $\Rightarrow$ TRUE
- [...] 7 IN (1,2,3,4,5) $\Rightarrow$ FALSE

- **ALL** prüft, ob ein Wert bezüglich eines Vergleichsoperators für alle Werte der Liste TRUE ergibt – nur dann wird der Ausdruck auch TRUE.

- [...] 1 <= ALL (1,2,3,4,5) $\Rightarrow$ TRUE
- [...] 5 <= ALL (1,2,3,4,5) $\Rightarrow$ FALSE

- **ANY** prüft, ob ein Wert bezüglich eines Vergleichsoperators für mindestens einen Werte der Liste TRUE ergibt – nur dann wird der Ausdruck auch TRUE.

- [...] 1 <= ANY (1,2,3,4,5) $\Rightarrow$ TRUE
- [...] 7 <= ANY (1,2,3,4,5) $\Rightarrow$ FALSE

Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte

Nullwerte

- Attribute können einen Nullwert **NULL** haben.
- Nullwerte entstehen
 - wenn kein Wert in der Datenbank vorhanden ist,
 - wenn der Wert vielleicht später nachgereicht wird,
 - im Zuge von Anfrageauswertung (bspw. äußere Joins)
- Nullwerte sind **Platzhalter für unterschiedliche Werte**.

Beispiel:

Student (ohne Nullwerte)

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
...	...	...
29555	Feuerbach	2

Student (mit Nullwerten)

MatrNr	Name	Semester
24002	Xenokrates	18
25403	NULL	12
26120	NULL	10
26830	Aristoxenos	NULL
...	...	...
29555	Feuerbach	NULL

Nullwerte sind nicht als Variable mit Name `NULL` zu verstehen.
 Insbesondere ist **(NULL = NULL) nicht wahr!**

Behandlung von Nullwerten

Arithmetische Ausdrücke

- Ergebnis **NULL**, sobald Nullwert in die Berechnung eingeht

`NULL + 1 = NULL;`

`NULL * 0 = NULL;`

Beispiel:

Student (mit Nullwerten)

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	18
25403	NULL	12
26120	NULL	10
26830	Aristoxenos	NULL
...	...	...
29555	Feuerbach	NULL

SELECT MatrNr, Name, Semester + 1
FROM Student;



Ergebnis

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	19
25403	NULL	13
26120	NULL	11
26830	Aristoxenos	NULL
...	...	...
29555	Feuerbach	NULL

Behandlung von Nullwerten

Vergleichsoperatoren

- SQL hat dreiwertige Logik: TRUE, FALSE, UNKNOWN. Das Resultat ist **UNKNOWN**, wenn mindestens eines der Argumente NULL ist.

Beispiel:

Student (mit Nullwerten)

MatrNr	Name	Semester
24002	Xenokrates	18
25403	NULL	12
26120	NULL	10
26830	Aristoxenos	NULL
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	NULL

```
SELECT COUNT (Semester)
FROM Student;
```

Ergebnis

COUNT(Semester)
6

```
SELECT COUNT (*)
FROM Student;
```

Ergebnis

COUNT(*)
8

Behandlung von Nullwerten

Vergleichsoperatoren

- SQL hat dreiwertige Logik: TRUE, FALSE, UNKNOWN. Das Resultat ist **UNKNOWN**, wenn mindestens eines der Argumente NULL ist.

Beispiel:

Student (mit Nullwerten)

MatrNr	Name	Semester
24002	Xenokrates	18
25403	NULL	12
26120	NULL	10
26830	Aristoxenos	NULL
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	NULL

```
SELECT COUNT (*)
FROM Student
WHERE Semester > 0 OR
      Semester <= 20;
```

Ergebnis

COUNT(*)
6

```
SELECT COUNT (*)
FROM Student;
```

Ergebnis

COUNT(*)
8

Behandlung von Nullwerten

Vergleichsoperatoren

- SQL hat dreiwertige Logik: TRUE, FALSE, UNKNOWN. Das Resultat ist **UNKNOWN**, wenn mindestens eines der Argumente NULL ist.

Beispiel:

Student (mit Nullwerten)

MatrNr	Name	Semester
24002	Xenokrates	18
25403	NULL	12
26120	NULL	10
26830	Aristoxenos	NULL
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	NULL

```
SELECT COUNT (*)
FROM Student
WHERE Semester = NULL;
```

Ergebnis

COUNT(*)
0

```
SELECT COUNT (*)
FROM Student <> NULL;
```

Ergebnis

COUNT(Semester)
0

Nullwerte

■ Behandlung von Nullwerten

- **Logische Ausdrücke**: werden nach den folgenden Tabellen berechnet:

AND	TRUE	UNKNOWN	FALSE
TRUE	TRUE	UNKNOWN	FALSE
UNKNOWN	UNKNOWN	UNKNOWN	FALSE
FALSE	FALSE	FALSE	FALSE

OR	TRUE	UNKNOWN	FALSE
TRUE	TRUE	TRUE	TRUE
UNKNOWN	TRUE	UNKNOWN	UNKNOWN
FALSE	TRUE	UNKNOWN	FALSE

NOT	
TRUE	FALSE
UNKNOWN	UNKNOWN
FALSE	TRUE

Aufgabe



- Welche Ergebnisse liefern die folgenden Ausdrücke?

Beispiel:

Gegeben: $A = 10$; $B = 20$; $C = \text{NULL}$

Ausdrücke:	$A < B \text{ OR } B < C$	TRUE
	$A > B \text{ AND } B > C$	FALSE
	$A > B \text{ OR } B > C$	UNKNOWN
	$\text{NOT } B = C$	UNKNOWN

Nullwerte

■ **WHERE-** bzw. **HAVING**-Bedingung:

- es werden nur Tupel weitergereicht, für die die Bedingung zu **TRUE** ausgewertet. Tupel, für die die Bedingung zu **UNKNOWN** ausgewertet, werden nicht ins Ergebnis aufgenommen.

■ Nullwerte werden abgefragt mittels:

- **IS NULL** bzw. **IS NOT NULL**

Beispiel:

```
SELECT Name, Semester  
FROM Student  
WHERE Semester IS NULL;
```

Beispiel:

```
SELECT Name, COALESCE(Semester, 0)  
FROM Student;  
-- COALESCE überprüft auf NULL-Wert und liefert statt dem  
-- leeren Wert (null) einen alternativen Wert zurück.
```

Nullwerte

■ Aggregatfunktionen:

- Ignorieren Nullwerte in den aggregierten Attributen.
- Ausnahme: **COUNT (*)** zählt die Anzahl der Zeilen in einer Tabelle.

Beispiel:

```
SELECT COUNT (SWS)
FROM Vorlesung;
```

Ergebnis

COUNT(SWS)
8

```
SELECT COUNT (*)
FROM Student;
```

Ergebnis

COUNT(*)
10

```
SELECT SUM (SWS)
FROM Vorlesung;
```

Ergebnis

SUM(SWS)
23

Vorlesung

VorlNr	Titel	SWS	GelesenVon
5001	Grundzüge	4	2137
5041	Ethik	NULL	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	NULL	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Nullwerte

■ Gruppierungen:

- **GROUP BY** betrachtet alle Nullwerte als wären sie identisch.
- Nullwerte in aggregierten Attributen werden als Gruppe zusammengefasst.



Beispiel:

R

A	B	C
1	NULL	100
1	NULL	200
NULL	NULL	300

Gruppiert nach den Attributen A und B ergibt die Gruppen

- `{[1, NULL, 100], [1, NULL, 200]}`
- `{[NULL, NULL, 300]}`

Nullwerte

■ Unteranfragen mit Nullwerten bei **IN**, **ALL**, **ANY**

● **IN**

- [...] 5 IN (1, NULL, 3, 4, 5) ⇒ **TRUE**
- [...] 7 IN (1, NULL, 3, 4, 5) ⇒ **UNKNOWN**
- [...] NULL IN (1, 2, 3, 4, 5) ⇒ **UNKNOWN**

● **ALL**: Tritt in der Liste ein NULL auf, wird das Ergebnis immer auf UNKNOWN gesetzt, wenn nicht vorher schon einer der Vergleiche ein FALSE liefert.

- [...] 1 <> ALL (1, 2, 3, 4, NULL) ⇒ **FALSE**
- [...] 1 <> ALL (NULL, 2, 3, 4, 5) ⇒ **UNKNOWN**

● **ANY**: Ergibt der Vergleich mit irgendeinem der Listenelemente TRUE, so wird der Ausdruck TRUE.

- [...] 1 >= ANY (1, 2, 3, 4, NULL) ⇒ **TRUE**

Wird kein Vergleich TRUE und es kommt ein Nullwert vor, ist das Ergebnis UNKNOWN.

- [...] 1 >= ANY (NULL, 2, 3, 4, 5) ⇒ **UNKNOWN**

Beispiel:

Professor

<u>PersNr</u>	...	...
2127	...	...
2137	...	...
2138	...	...

Vorlesung

<u>VorlNr</u>	...	gelesenVon
5001	...	2137
5002	...	2138
5041	...	NULL

Beispiel:

Professor

<u>PersNr</u>	...	...
2127	...	...
2137	...	...
2138	...	...

Vorlesung

<u>VorlNr</u>	...	gelesenVon
5001	...	2137
5002	...	2138
5041	...	NULL

```
SELECT * FROM Professor WHERE PersNr NOT IN  
(SELECT gelesenVon FROM Vorlesung); -- Professoren ohne Vorlesungen
```


Beispiel:

Professor

PersNr	...	...
2127	...	...
2137	...	...
2138	...	...

Vorlesung

VorlNr	...	gelesenVon
5001	...	2137
5002	...	2138
5041	...	NULL

```
SELECT * FROM Professor WHERE PersNr NOT IN
(SELECT gelesenVon FROM Vorlesung); -- Professoren ohne Vorlesungen
```

Umwandlung:

```
SELECT * FROM Professor WHERE PersNr NOT IN
(2137, 2138, NULL);
```

Beispiel:

Professor

PersNr	...	...
2127	...	...
2137	...	...
2138	...	...

Vorlesung

VorlNr	...	gelesenVon
5001	...	2137
5002	...	2138
5041	...	NULL

```
SELECT * FROM Professor WHERE PersNr NOT IN
(SELECT gelesenVon FROM Vorlesung); -- Professoren ohne Vorlesungen
```

Umwandlung:

```
SELECT * FROM Professor WHERE PersNr NOT IN
(2137, 2138, NULL);
```

Umwandlung mit Vergleichsoperatoren:

```
SELECT * FROM Professor WHERE
(PersNr <> 2137) AND (PersNr <> 2138) AND (PersNr <> NULL);
```

Beispiel:

Professor

PersNr	...	...
2127	...	...
2137	...	...
2138	...	...

Vorlesung

VorlNr	...	gelesenVon
5001	...	2137
5002	...	2138
5041	...	NULL

```
SELECT * FROM Professor WHERE PersNr NOT IN
(SELECT gelesenVon FROM Vorlesung); -- Professoren ohne Vorlesungen
```

Umwandlung:

```
SELECT * FROM Professor WHERE PersNr NOT IN
(2137, 2138, NULL);
```

Umwandlung mit Vergleichsoperatoren:

```
SELECT * FROM Professor WHERE
(PersNr <> 2137) AND (PersNr <> 2138) AND (PersNr <> NULL);
```

für PersNr=2127 ergibt **TRUE AND TRUE AND UNKNOWN -> UNKNOWN**
d.h. Tupel PersNr = 2127 wird nicht selektiert!

Beispiel:

Professor

PersNr	...	...
2127	...	...
2137	...	...
2138	...	...

Vorlesung

VorlNr	...	gelesenVon
5001	...	2137
5002	...	2138
5041	...	NULL

```
SELECT * FROM Professor WHERE PersNr NOT IN
(SELECT gelesenVon FROM Vorlesung); -- Professoren ohne Vorlesungen
```

Umwandlung:

```
SELECT * FROM Professor WHERE PersNr NOT IN
(2137, 2138, NULL);
```

Umwandlung mit Vergleichsoperatoren:

```
SELECT * FROM Professor WHERE
(PersNr <> 2137) AND (PersNr <> 2138) AND (PersNr <> NULL);
```

für PersNr=2127 ergibt **TRUE AND TRUE AND UNKNOWN -> UNKNOWN**
d.h. Tupel PersNr = 2127 wird nicht selektiert!

Lösung wäre bspw.:

```
SELECT * FROM Professor WHERE PersNr NOT IN
(SELECT gelesenVon FROM Vorlesung WHERE gelesenVon IS NOT NULL);
```

Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte

Spezielle Sprachkonstrukte

Erweiterungen des SFW-Blocks

- Innerhalb der **SELECT**-Klausel
 - Arithmetische Ausdrücke
 - Fallunterscheidungen – **CASE**
- Innerhalb der **FROM**-Klausel
 - Äußerer Verbund – **OUTER JOIN**
 - Linker äußerer Verbund – **LEFT OUTER JOIN**
 - Rechter äußerer Verbund – **RIGHT OUTER JOIN**
- Innerhalb der **WHERE**-Klausel
 - Bereichsselektion – **BETWEEN**
 - Ähnlichkeitsselektion – **LIKE**
- Benannte Anfragen – **WITH**

Bereichsselektion

Notation:

Attribut [**NOT**] **BETWEEN** Konstante1 **AND** Konstante2

ist Abkürzung für

Attribut $\geq$ Konstante1 **AND**
Attribut $\leq$ Konstante2

- Schränkt damit Attributwerte auf das abgeschlossene Intervall [Konstante1, Konstante2] ein

Beispiel:

Suchen Sie jene Studierenden, die zwischen dem ersten und dem vierten Semester inskribiert sind.

```
SELECT *  
FROM Student  
WHERE Semester BETWEEN 1 AND 4;
```

Ähnlichkeitsselektion 1/5

- Selektion nach Tupel
 - von denen **nur Teile** des Inhalts **bekannt** sind oder
 - die einem vorgegeben **Selektionskriterium** möglichst nahe kommen
- Anwendung bei **ähnlichen Zeichenketten**, die nicht vollständig identisch sind (z.B. "Meyer" u. "Meier")
 - ⇒ **"Unscharfe Suche"**

Notation:

WHERE Attribut [**NOT**] **LIKE** Spezialkonstante

- Spezialkonstante (= **Muster**) kann die Sonderzeichen (= Maskierungszeichen) '%' und '_' enthalten
 - '%' steht für kein oder beliebig viele Zeichen
 - '_' steht für genau ein Zeichen

Ähnlichkeitsselektion 2/5

Beispiel:

Suchen Sie die Matrikelnummer von Theophrastos, wobei Sie nicht wissen, ob er sich mit 'h' schreibt:

```
SELECT *  
FROM Student  
WHERE Name LIKE 'T%eophrastos';
```

Suchen Sie die Matrikelnummern jener Studenten, die mindestens eine Vorlesung über Ethik gehört haben:

```
SELECT DISTINCT MatrNr  
FROM Vorlesung NATURAL JOIN hoeren  
WHERE Titel LIKE '%thik';
```

Ähnlichkeitsselektion 3/5

- Die Zeichen '%' und '_' werden in Vergleichszeichenketten als Sonderzeichen interpretiert.
- Durch Verwendung der **ESCAPE**-Klausel kann auch nach Sonderzeichen wie nach „normalen“ Zeichen gesucht werden.
- In der **ESCAPE**-Klausel kann ein spezielles **ESCAPE**-Zeichen frei definiert werden, welches dann vorangestellt wird.

Beispiel:

Suchen Sie alle Telefonnummern, die folgendes Muster aufweisen: sie beginnen mit einem beliebigen Zeichen, gefolgt von '2%', dann gefolgt von beliebig vielen weiteren Zeichen:

```
SELECT Telefonnummer  
FROM Student  
WHERE Telefonnummer LIKE '_2/%%' ESCAPE '/';
```

Ähnlichkeitsselektion 4/5

- **LIKE** 'kennt' keine Wortgrenzen. Der gesamte **CHAR**-Attributwert wird **als eine Zeichenkette** (quasi als ein Wort) aufgefasst, einschließlich aller darin auftretenden Leerzeichen, Zahlen und Sonderzeichen.
- Möglichkeit, präzise Anfragen auf **CHAR**-Attribute zu formulieren, ist dadurch eingeschränkt.
- SQL:2003: **SIMILAR TO**-Prädikat mit regulären Ausdrücken*
- Viele **DBMS** bieten (oft proprietäre) **Zusatzfunktionen** an und unterstützen **reguläre Ausdrücke** als Suchmuster.

* Standard Regular Expressions: https://de.wikipedia.org/wiki/Regul%C3%A4rer_Ausdruck

Ähnlichkeitsselektion 5/5

Notation (Oracle):

REGEXP_LIKE	stellt fest, ob ein Muster in der Zeichenkette existiert.
REGEXP_SUBSTR	extrahiert die zum regulären Ausdruck passende (Teil-)Zeichenkette.
REGEXP_INSTR	gibt die Zeichenposition zurück, an der die zum Ausdruck passende Teilzeichenkette beginnt.
REGEXP_REPLACE	ersetzt die zum Ausdruck passende Teilzeichenkette durch eine andere.

Beispiel (Oracle):

Suchen Sie alle Studierenden mit dem Namen 'Steven' oder 'Stephen'?

```
SELECT MatrNr, Name  
FROM Student  
WHERE REGEXP_LIKE (Name, '^Ste(v|ph)en$');
```

Arithmetische Ausdrücke 1/3

- Arithmetische Operationen/Funktionen auf verschiedene Datentypen:
 - Numerische Wertebereiche
 - etwa +, -, * und /
 - Strings
 - Operationen wie `char_length` (aktuelle Länge eines Strings)
 - Verkettungsoperation `||`
 - Operation `substring` (Suchen einer Teilzeichenkette an bestimmten Positionen des Strings)
 - Datumstypen, Zeitintervalle, Zeitangaben
 - Operationen wie `current_date` (aktuelles Datum)
 - `current_time` (aktuelle Zeit)
 - +, -

Arithmetische Ausdrücke 2/3

Verkettungsoperator

Funktionsaufruf:
Wertumwandlung

Beispiel:

```
SELECT 'Vorlesungsnummer: ' || to_char(VorlNr) AS Vorlesung,
       Titel, SWS * 14 AS Präsenzstunden
FROM Vorlesung;
```

Spalten-Aliasnamen

Arithmetischer Operator



Ergebnis

VORLESUNG	TITEL	PRÄSENZSTUNDEN
Vorlesungsnummer: 5001	Grundzüge	56
Vorlesungsnummer: 5041	Ethik	56
Vorlesungsnummer: 5043	Erkenntnistheorie	42
Vorlesungsnummer: 5049	Mäeutik	28
Vorlesungsnummer: 4052	Logik	56
Vorlesungsnummer: 5052	Wissenschaftstheorie	42
Vorlesungsnummer: 5216	Bioethik	28
Vorlesungsnummer: 5259	Der Wiener Kreis	28
Vorlesungsnummer: 5022	Glaube und Wissen	28
Vorlesungsnummer: 4630	Die 3 Kritiken	56

Arithmetische Ausdrücke 3/3

- Eine arithmetische Operation (+, -, *, /) mit einem Nullwert führt auf einen Nullwert

Vorlesungen

<u>VorlNr</u>	<u>Titel</u>	<u>SWS</u>	<u>GelesenVon</u>
5001	Grundzüge	4	2137
5041	Ethik	NULL	2125
...	...	...	...

Beispiel:

```
SELECT VorlNr, SWS * 14 AS Präsenzstunden
FROM Vorlesung;
```



Ergebnis

<u>VorlNr</u>	<u>PRÄSENZSTUNDEN</u>
Vorlesungsnummer: 5001	56
Vorlesungsnummer: 5041	NULL
...	...

Fallunterscheidung

Beispiel:

Wandeln Sie die Prüfungsnoten in Werte der Studienabteilung um:

```
SELECT MatrNr, (  
    CASE  
        WHEN Note < 1.5 THEN 'S1'  
        WHEN Note < 2.5 THEN 'G2'  
        WHEN Note < 3.5 THEN 'B3'  
        WHEN Note < 4.5 THEN 'G4'  
        ELSE 'N5' END) AS Note  
FROM pruefen;
```



Ergebnis

MatrNr	Note
28106	S1
25403	G2
27550	G2

Verbunde

- SQL unterstützt folgende (explizite) **JOIN-Schlüsselwörter**:
 - **CROSS JOIN**: Kreuzprodukt
 - **NATURAL JOIN**: Natürlicher Verbund
 - [**INNER**] **JOIN**: Innerer Verbund (oder Theta Join)
 - (**LEFT** | **RIGHT** | **FULL**) **OUTER JOIN**: Äußerer Verbund

CROSS JOIN: Professor **x** Vorlesung

```
SELECT *  
FROM Professor CROSS JOIN Vorlesung;
```

Natürlicher Verbund

- Die Werte jener Spalten, deren Attributnamen dieselben sind, werden gleichgesetzt.

NATURAL JOIN: Professor ⋈ Vorlesung

Geben Sie Name und Matrikelnummer der Studierenden und den Titel der von ihnen gehörten Vorlesungen aus.

```
SELECT MatrNr, Name, Titel  
FROM Student NATURAL JOIN hoeren NATURAL JOIN Vorlesung;
```

Innerer (allgemeiner) Verbund

INNER JOIN: Professor $\bowtie_{\theta}$ Vorlesung

Welche Professoren lesen die LVA Mäeutik?

```
SELECT Name, Titel
FROM Professor INNER JOIN Vorlesung
ON (PersNr = gelesenVon AND Titel = 'Maeeutik');
```

Äußere Verbunde 1/3

- Zusätzlich zu klassischen Verbund (**INNER JOIN**)
 - seit SQL2 auch äußerer Verbund
 - Übernahme von Tupel in das Ergebnis, die **keine Join-Partner** in der jeweiligen anderen Relation haben und
 - Auffüllen mit **NULL-Werten**
- **OUTER JOIN**
 - übernimmt alle Tupel beider Operanden verlustfrei
 - Langfassung: **NATURAL FULL OUTER JOIN**
- **LEFT OUTER JOIN**
 - übernimmt alle Tupel des linken Operanden verlustfrei
- **RIGHT OUTER JOIN**
 - übernimmt alle Tupel des rechten Operanden verlustfrei

Äußere Verbunde 2/3

LINKS

A	B
1	2
2	3

RECHTS

B	C
3	4
4	5

NATURAL JOIN

A	B	C
2	3	4



```
SELECT * FROM LINKS NATURAL JOIN RECHTS;
```

FULL OUTER JOIN

A	B	C
1	2	NULL
2	3	4
NULL	4	5

SELECT *

```
FROM LINKS NATURAL FULL OUTER JOIN RECHTS;
```

LEFT OUTER JOIN

A	B	C
1	2	NULL
2	3	4

SELECT L.A, L.B, R.C

```
FROM LINKS L LEFT OUTER JOIN RECHTS R ON L.B=R.B;
```

RIGHT OUTER JOIN

A	B	C
2	3	4
NULL	4	5

SELECT L.A, L.B, R.C

```
FROM LINKS L RIGHT OUTER JOIN RECHTS R ON L.B=R.B;
```

Äußere Verbunde 3/3

LEFT OUTER JOIN: Student ⋈ prüfen

Geben Sie eine Liste **aller** Studierenden aus und ihre Noten zu den abgeprüften Vorlesungen.

```
SELECT s.MatrNr, s.Name, COALESCE(p.VorlNr, 0), COALESCE(p.Note, 0)
FROM Student s LEFT OUTER JOIN prüfen p
ON s.MatrNr = p.MatrNr;
```



Student ⋈ prüfen

MatrNr	Name	VorlNr	Note
24002	Xenokrates	0	0
25403	Jonas	5041	2
26120	Fichte	0	0
26830	Aristoxenos	0	0
27550	Schopenhauer	4630	2
28106	Carnap	5001	1
29120	Theophrastos	0	0
29555	Feuerbach	0	0

Beispiel ohne Verwendung des OUTER JOINS:

```
SELECT s.MatrNr, s.Name, p.VorlNr, p.Note
FROM Student s INNER JOIN prüfen p
ON s.MatrNr = p.MatrNr;
```



Student ⋈ prüfen

MatrNr	Name	VorlNr	Note
25403	Jonas	5041	2
27550	Schopenhauer	4630	2
28106	Carnap	5001	1

Benannte Anfragen 1/2

- Anfrageausdruck, der in der Anfrage mehrfach referenziert werden kann

Notation:

WITH anfrage-name[attribut-liste] **AS** (anfrage-audruck)

- Vorteile:
 - Verbessert die Lesbarkeit der Anfrage
 - Verbessert oft die Performance, da Klausel nur einmal ausgewertet wird
- Nachteile:
 - **WITH**-Unterabfragen dürfen nicht korreliert sein
 - **WITH**-Tabelle nur für den Kontext der Abfrage verfügbar

Benannte Anfragen 2/2

Anfrage ohne WITH:

Geben Sie Vorlesungsnummer und Lehranteil für alle Vorlesungen der Fakultät aus (~quasi die Einschaltquoten im Fernsehen).

```
SELECT h.VorlNr, h.AnzProVorl, g.GesamtAnz,
       h.AnzProVorl/g.GesamtAnz AS Lehranteil
FROM   ( SELECT VorlNr, COUNT(*) AS AnzProVorl
        FROM hoeren
        GROUP BY VorlNr) h CROSS JOIN
        ( SELECT COUNT(*) AS GesamtAnz
        FROM Student) g;
```

VorlNr	AnzProVorl	GeamtAnz	Lehranteil
4052	1	8	0.125
5001	4	8	0.5
5022	2	8	0.25
...	...	...	...

Anfrage mit WITH:

```
WITH h AS ( SELECT VorlNr, COUNT(*) AS AnzProVorl
            FROM hoeren
            GROUP BY VorlNr),
     g AS ( SELECT COUNT(*) AS GesamtAnz
            FROM Student)
SELECT h.VorlNr, h.AnzProVorl, g.GesamtAnz,
       h.AnzProVorl/g.GesamtAnz AS Lehranteil
FROM   h CROSS JOIN g;
```


Überblick

- Einführung Datenabfragesprache SQL
- Datenabfragesprache SQL
 - SQL-Kern
 - Einfache SQL-Anfragen
 - Anfragen über mehrere Relationen
 - Mengenoperationen
 - Aggregatfunktionen
 - Aggregate und Gruppierung
 - Geschachtelte Anfragen
 - Existenziell quantifizierte Anfragen
 - Allquantifizierte Anfragen
 - Nullwerte
 - Spezielle Sprachkonstrukte



Zusammenfassung

- **SQL** ist die in der Praxis am **weitesten verbreitete** Datenbanksprache für relationale Datenbanken.
- **SQL-Kern** mit Bezug zur **Relationenalgebra**
- Der Sprachumfang von SQL ist einer **permanenten Weiterentwicklung** und Standardisierung unterworfen.
- **Auf dem Markt** befindliche **Datenbanksysteme** realisieren **weitgehend** die im Standard **SQL2** definierten Konzepte und bereits viele Konzepte der Standards SQL:1999, SQL:2003, SQL:2008, SQL:2011, SQL:2016 und SQL:2023.
- Viele Hersteller wie Oracle, Microsoft oder IBM haben in ihren Systemen **über den Standard hinausgehende SQL-Erweiterungen** implementiert.
- Datenbankhersteller messen sich anhand der angebotenen Ausdrücke und Prädikate (sowohl in **Funktionalität** als auch **Geschwindigkeit**)